

Offloaded Asynchronous Semantic Segmentation from Embedded Clients Using OpenVINO Servers

Marko Mihajlovic

Abstract

Semantic segmentation – the pixel-wise classification of images – is a critical computer vision task for robotics and embedded systems, but it is computationally heavy. Resource-constrained devices like the Raspberry Pi struggle to run modern segmentation models at real-time speeds, often achieving only a few frames per second (FPS) due to limited CPU performance[1]. This paper presents a system architecture that offloads semantic segmentation from an embedded client to a more powerful server equipped with Intel’s OpenVINO toolkit. The proposed design enables *asynchronous* operation: the embedded client continuously captures and streams camera frames (after lightweight compression), while a laptop-class server performs segmentation inference in parallel and returns results without blocking the client. We detail the end-to-end pipeline, including the Raspberry Pi client design (camera capture, JPEG encoding, network framing, and in-flight frame management) and the OpenVINO-based server (multi-process inference, result aggregation, and back-pressure control). A custom binary communication protocol is developed to efficiently pair requests and responses, supporting multiple payload variants such as raw segmentation labels and visualization overlays. We also describe post-processing techniques including projecting segmentation outputs into a bird’s-eye-view (BEV) perspective and adding custom “flower probe” markers to assist in visualization and fusion of results over time. Experimental evaluation demonstrates that our offloaded asynchronous approach achieves significantly higher throughput and lower latency compared to on-board processing. In a prototype, the system attained near-real-time performance (on the order of tens of FPS with sub-100ms round-trip latency) for segmentation of 480p video frames, vastly outperforming the ~1–2 FPS achievable on the Raspberry Pi’s CPU[1]. We discuss observations on network utilization, the impact of asynchronous pipelining on overall frame rate, and the division of labor between the Pi and the laptop. Finally, we outline current limitations – such as network dependence – and future directions including multi-client scalability, integration of hardware accelerators, and improved fault tolerance. The results suggest that offloading semantic segmentation to an edge server using an asynchronous pipeline is a promising solution for deploying advanced vision capabilities on low-power robotic platforms.

Introduction and Motivation

Semantic segmentation provides a dense understanding of scene content by assigning a class label to each pixel in an image. This capability is invaluable in robotics and autonomous systems for tasks like environment mapping, obstacle detection, and scene

understanding. However, state-of-the-art segmentation models are computationally intensive, typically involving deep convolutional neural networks or transformers with millions of parameters. Running such models in real time on embedded devices is challenging because of limited processing power and memory. For example, the CPU of a Raspberry Pi can barely sustain more than a few frames per second on modern segmentation tasks[1]. This performance is insufficient for applications like mobile robots or drones that require fast perception updates.

One approach to address this gap is to leverage **edge computing**, where heavy computation is offloaded from the resource-constrained device to a more powerful nearby server (the “edge node”). Unlike cloud computing which might introduce high latency, edge computing places computation close to the data source (e.g., on the same local network), enabling low-latency processing. Indeed, prior work in intelligent vehicles and robots has highlighted that offloading intensive tasks such as image segmentation to proximate edge servers can overcome onboard limitations and avoid performance bottlenecks[2][3]. Onboard computing power often cannot keep pace with the demands of high-precision vision tasks like semantic segmentation, leading to severe frame rate and latency issues[2]. By shifting the heavy lifting to an external server, an embedded client can benefit from faster processing and potentially run more complex models than it could locally.

In addition to raw processing capability, **asynchronous pipeline design** is crucial for maximizing throughput. Rather than a sequential request-wait-response pattern, an asynchronous approach allows the client and server to work in parallel. The key idea is that when one frame is being processed on the server, the client can already be capturing or preparing the next frame, instead of idly waiting for results. This overlaps computation with communication and sensor capture, effectively pipelining the tasks. The advantage is analogous to the async inference support in frameworks like OpenVINO, where the CPU or accelerator can be kept busy while the application queues up new work[4]. By not blocking on each inference, the system can achieve a higher overall frame rate than the per-frame latency would suggest. This design also helps prevent the camera feed from stalling – new images are continuously streamed, which is important for maintaining a smooth video flow from the robot’s perspective.

To implement the offload strategy, we build on the **Intel OpenVINO** toolkit at the server. OpenVINO is an optimized inference engine for deep learning models, designed to run efficiently on Intel hardware including CPUs, integrated GPUs, VPUs (like the Movidius Neural Compute Stick), and FPGAs[5][6]. It provides optimizations such as layer fusion and uses hardware-specific instructions, enabling faster inference of neural networks without needing a dedicated GPU. By using OpenVINO on a laptop (with an Intel CPU in our case), the server can perform segmentation significantly faster than the Raspberry Pi’s native capabilities. For instance, OpenVINO can accelerate models from popular frameworks (TensorFlow, PyTorch, etc.) on Intel CPUs and even leverage multi-core and vector instructions for speedup[6]. This makes it feasible to run complex segmentation models at inference speeds suitable for real-time robotics.

In this paper, we present **Offloaded Asynchronous Semantic Segmentation**, a system where a Raspberry Pi (embedded client) offloads live camera frames to a laptop (server) that performs semantic segmentation using OpenVINO and returns the results. The system is designed with **modularity and efficiency** in mind: the client handles sensor capture and data transmission, while the server handles model inference and result generation. A lightweight binary protocol is developed to coordinate the exchange, supporting multiple response formats (either raw segmentation or visualization). We also incorporate post-processing steps at the server to transform and augment the segmentation results (e.g., projecting them into a bird’s-eye view map) before sending back to the client. The **motivation** for this architecture is to enable advanced vision-based autonomy on platforms that cannot natively support it, by tapping into nearby computational resources without incurring the high latency of cloud computing. This is particularly relevant for scenarios such as indoor service robots or autonomous drones in IoT environments, where a local edge computer can be available (a base station or companion laptop) to assist the low-power agent.

To summarize, our contributions include: (1) an end-to-end system architecture for real-time semantic segmentation offloading from an embedded device to an edge server; (2) a custom asynchronous communication protocol that allows multiple in-flight frames and out-of-order results through unique pairing IDs; (3) design details for both the Raspberry Pi client and the OpenVINO server, such as efficient frame compression, multi-process parallel inference, and back-pressure handling; (4) a post-processing module that converts segmentation outputs into a more robot-friendly perspective (BEV) and introduces “flower probes” for enhanced visualization and analysis; and (5) an experimental evaluation of the system’s performance, along with a discussion of observations, limitations, and future improvements. The remainder of this paper is organized as follows. **Section II (Related Work)** reviews existing approaches to edge offloading and semantic segmentation on embedded platforms. **Section III (System Architecture Overview)** gives a high-level description of our system’s components and data flow. **Section IV (Client Design)** and **Section V (Server Design)** detail the implementation of the Raspberry Pi client and the OpenVINO server, respectively. **Section VI (Communication Protocol Design)** delineates the custom message format and pairing mechanism. **Section VII (Postprocessing and Visualization)** describes how results are transformed into a BEV map and fused with probes. **Section VIII (Performance and Observations)** presents measured results and insights. **Section IX (Limitations and Future Directions)** discusses the current system’s constraints and potential enhancements. Finally, **Section X (Conclusion)** concludes the paper.

Related Work

Real-time image processing on resource-limited devices has long been a challenge in computer vision, and various strategies have been explored to address it. **Cloud Robotics and Edge Offloading:** In the context of robotics and IoT, a prominent strategy is to offload computation to external servers – either cloud data centers or nearer edge computers. Early cloud robotics architectures envisioned robots delegating heavy

computation to cloud services, but network latency and reliability issues can impair real-time tasks. Recent trends favor *edge computing*, where computation is performed on infrastructure closer to the robot (for example, on a local PC or edge cluster on the same network). Edge computing can drastically reduce latency and bandwidth usage compared to remote cloud processing[7][8]. Zou et al. (2023) specifically highlight that offloading computationally intensive tasks like semantic image segmentation and sensor fusion to edge nodes is a key promise of the edge computing paradigm for connected systems[3]. By moving perception and AI workloads to nearby servers, systems can overcome the limited onboard resources and achieve timely processing[2]. Our work follows this edge-offloading approach, focusing on a single-robot (single-client) scenario with a dedicated local server.

Several research efforts have demonstrated offloading in robotics and automotive domains. For example, Ben Ali et al. partitioned the visual SLAM (Simultaneous Localization and Mapping) algorithm by keeping light-weight tracking on the robot while offloading heavy map-building and loop-closure tasks to an edge server[9]. This split architecture maintained accuracy while reducing the computational and memory burden on the robot[9]. In autonomous driving, DNN-based perception like object detection or segmentation is often offloaded to roadside edge units to meet real-time requirements that in-vehicle computers struggle with[10]. **EdgeSLAM** and **Edge-assisted perception** frameworks commonly set up a pipeline where sensor data (camera images, LIDAR scans, etc.) are transmitted over Wi-Fi or dedicated links to an edge computer that runs heavy algorithms, returning distilled results (e.g., object IDs, maps) to the robot. These works validate that offloading can enable complex vision capabilities on robots that would otherwise be infeasible to run onboard. Our system can be seen as a specific instantiation of this concept for semantic segmentation: the Raspberry Pi acts as an IoT/robot node streaming camera data, and the laptop is the edge processor delivering segmentation outcomes back to the Pi in real time.

Asynchronous and Real-Time Streaming: Prior systems emphasize the importance of asynchronous operation to maintain high throughput. In an ideal scenario, the sensor data pipeline and the inference pipeline run in parallel. For instance, some IoT video analytics frameworks use producer–consumer models where frames are continuously produced by a camera and a queue of edge workers consume and analyze them, ensuring the camera is never idle waiting on analysis. The asynchronous processing approach is also exemplified in some practical tutorials: a demonstration by Alexey Stern (2020) showed that by offloading video capture and rendering to a client and running inference on a separate server thread, a Raspberry Pi could serve a smooth 30 FPS video stream while simultaneously performing object detection on each frame[11]. Although that system had the Pi doing inference (with a Movidius NCS accelerator) and a web client doing display, the principle carries over – decoupling the frame production from the inference consumption leads to better overall frame rates and user experience. Our design explicitly incorporates an asynchronous, multi-frame pipeline so that the Pi’s camera capture loop is not blocked by the segmentation

processing time. This is crucial for robotics applications where fresh sensor data should be acquired continuously, even if processing lags by a few frames.

Hardware Acceleration vs. Offloading: Another line of related work is the use of hardware accelerators on the edge device itself. Instead of offloading to a separate machine, one can attach specialized inference hardware to the embedded device. For example, the Intel Movidius Neural Compute Stick (NCS2) is a USB dongle that contains a low-power Myriad VPU optimized for deep learning tasks. It has been shown to significantly speed up inference on devices like the Raspberry Pi when using frameworks like OpenVINO[12]. Rosebrock (2019) demonstrated using OpenVINO with an NCS on the Raspberry Pi to achieve real-time object detection, noting that without such acceleration the Pi's CPU is the bottleneck for deep nets[13][12]. Similarly, Google's Edge TPU (Coral) and NVIDIA's Jetson platform are popular solutions for on-device acceleration. These approaches reduce dependency on external connectivity and can work offline. However, they have limitations: the added hardware increases cost and complexity, and may still fall short for very large models or high resolutions. Offloading to a generic server offers flexibility (the server can have a high-end GPU/CPU and can be upgraded independently) and can handle multiple clients or other services concurrently. In our scenario, we opted for offloading to utilize a standard laptop CPU with OpenVINO optimizations, avoiding the need for specialized accelerators on the Pi. Nonetheless, our approach is complementary – in future one could incorporate an accelerator on the client to pre-process or run a smaller model and offload only what is needed, further optimizing the pipeline.

Communication Protocols for Vision Offloading: Efficiently streaming video or images is a well-studied problem. Protocols like RTSP or MJPEG streaming send continuous frames which can be received and processed by a server or client. In robotics (especially those using ROS – Robot Operating System), it is common to send image frames over network using publish/subscribe middleware, but that usually involves overhead (e.g., ROS messages, serialization) and typically operates in a synchronous request/reply manner if a service call is used. Some systems use lightweight protocols (raw UDP video streams or custom TCP protocols) to minimize latency. Our approach uses a custom TCP-based protocol tailored for the segmentation task, which ensures reliable delivery and uses binary framing for low overhead. We chose TCP for reliability (no frame loss) and ease of implementation (built-in ordering), acknowledging that well-engineered UDP protocols could further cut latency at the cost of complexity. We also compress each frame as JPEG to drastically reduce the data size per frame, which is a standard technique in streaming – raw uncompressed frames would consume bandwidth quickly (on the order of multiple megabytes per frame for modest resolutions). By using JPEG at a reasonable quality setting, we send on the order of tens of kilobytes per frame, which a typical wireless or wired LAN can handle at dozens of FPS. Prior works have similarly used image compression when offloading video data to mitigate network load[14] (some even train models on compressed domain). Unlike video streaming protocols that aim for continuous one-way flow, our protocol is a two-way RPC style (send image, get result), which required designing a framing format

with request and response pairing. This draws inspiration from remote procedure call systems and other vision RPC frameworks (like AWS Robomaker or custom gRPC for image analysis), but we implemented a simpler bespoke solution optimized for our use case.

In summary, the literature suggests that moving computation to more capable devices (edge or cloud) is a viable way to enable advanced perception on small robots. It also emphasizes careful handling of communication to avoid undue delays, and in some cases, the use of asynchronous pipelines to maximize throughput. Our work contributes to this landscape by providing a detailed case study of an offloaded semantic segmentation system, highlighting the specific design choices and engineering trade-offs in making it work in practice for embedded clients. We also integrate additional features like BEV projection of segmentation, which connects to multi-sensor fusion research that often converts camera understanding into a unified spatial map[15][14]. Our system can be seen as an example of how edge computing can be harnessed in a practical robotics scenario, complementing both research prototypes and real-world deployment considerations (such as network reliability and system robustness).

System Architecture Overview

The proposed system consists of two main components – the **embedded client (Raspberry Pi)** and the **edge server (Laptop)** – connected via a network link (typically a local Ethernet or Wi-Fi). At a high level, the Raspberry Pi continuously captures images from a camera and sends them to the laptop, which performs semantic segmentation on each image and sends back the results to the Pi. The design is *streaming and asynchronous*: multiple frames can be in various stages of processing concurrently, forming a pipeline that keeps all components busy and maximizes throughput. Figure 1 (to be provided) will illustrate the overall architecture and data flow between client and server.

Processing Pipeline: The end-to-end workflow can be summarized in the following sequence of steps:

1. **Frame Capture (Client):** The Raspberry Pi's camera module (or a USB camera) captures an image frame. This can be done at a target resolution (e.g., 640×480) and frame rate (e.g., 15–30 FPS) suitable for the task and network bandwidth. The capture is typically done using a library like OpenCV or the Pi's camera SDK, producing a raw image in memory.
2. **Local Preprocessing and Compression (Client):** The captured frame is optionally preprocessed (for example, slight resizing or normalization) and then compressed into a JPEG format. JPEG compression significantly reduces the data size of the image (by an order of magnitude or more) while retaining essential visual information for segmentation. This step prepares the image data for efficient transmission over the network.

3. **Packet Framing and Transmission (Client):** The client wraps the JPEG image bytes into a custom **binary message frame** that includes metadata like a frame ID and payload length (details in Section “Communication Protocol Design”). The framed message is then sent over a TCP socket to the server. The client does not pause after sending; it immediately proceeds to capture the next frame (unless flow control dictates otherwise), enabling a continuous stream of outgoing frames.
4. **Network Transfer:** The image packet travels over the network to the server. In a typical LAN environment, this transfer introduces some latency (e.g., a few milliseconds for local Ethernet, potentially a bit more for Wi-Fi depending on signal quality). The TCP protocol ensures the data arrives intact and in order, but since we allow multiple outstanding frames, the “in order” property refers to data within each message – not that responses must come back in the same order as requests (our protocol handles out-of-order returns via IDs).
5. **Reception and Decoding (Server):** The server’s network listener receives the incoming bytes, reassembles the message based on the framing protocol, and extracts the JPEG image and its ID. The JPEG is then decoded back into an image matrix (e.g., using OpenCV’s `imdecode` or a similar function) to recover the raw pixel data required for the neural network.
6. **Inference (Server):** The server runs the semantic segmentation model on the decoded image. This step is executed by the OpenVINO inference engine, which takes the image as input and produces a segmentation output (typically a 2D array of class labels for each pixel, or a multi-channel array if logits/probabilities are produced and need post-processing). We design the server to handle this step in parallel for multiple frames: a pool of worker processes (or threads) may be utilized so that several inferences can happen concurrently on multi-core hardware. Each inference is associated with the frame ID it is processing.
7. **Post-processing and Result Packaging (Server):** Once a frame’s segmentation result is ready, the server post-processes the output according to the requested payload type. We have developed multiple payload variants:
 8. If the client expects **raw class IDs** (“ids”), the server will package the segmentation map (the label for each pixel) in an efficient format. This could be a run-length encoded byte array or a compressed image (e.g., PNG where each pixel’s color represents a class).
 9. If the client expects a **visualization** (“viz”), the server will overlay or combine the segmentation with the original image (for example, by coloring each segment with a distinct transparent color or drawing class outlines) to produce an annotated image. This annotated image is then encoded (e.g., as JPEG or PNG) for transmission.

10. A third mode (“jpg”) can be a baseline where the server simply returns the original image (echo) or a similarly sized payload, used mainly for debugging or measuring base latency with minimal processing.

Additionally, as part of post-processing, the server may perform the BEV projection and add special markers (“flower probes”) to the visualization (see Section “Postprocessing (viz)” for details).

1. **Response Transmission (Server):** The server wraps the result (segmentation data or visualization image) into a response message with the corresponding frame ID and type code. This message is sent back over the socket to the client. The server then can immediately proceed to the next pending frame in its queue (if any), or wait for new frames if the pipeline is momentarily empty.
2. **Result Reception (Client):** The Raspberry Pi client receives the response message from the server, identifies which frame ID the result corresponds to, and then processes the result. If it’s a visualization image, the client can directly display or save it. If it’s raw segmentation data, the client might apply its own visualization (e.g., coloring and displaying on a screen) or utilize the data for higher-level decision-making (such as obstacle avoidance logic in a robot). The client then frees that frame’s slot, allowing another frame to be sent if it was holding off due to too many in flight.

This pipeline allows continuous operation. At any given time, the camera might be capturing a new frame (step 1) while previous frames are being encoded (step 2), transmitted (step 3-4), processed (step 6), or awaiting return (step 7-9). The asynchronous design ensures that the overall throughput is not bottlenecked by any single component; instead, it will be governed by the slowest stage, but other stages will overlap to utilize any idle time.

Asynchronous In-Flight Control: A crucial aspect of the architecture is managing how many frames are “in flight” (i.e., have been sent to the server but not yet returned). Too many in-flight frames could lead to high latency (frames queued up at the server) and memory usage spikes, whereas too few in-flight frames means under-utilization of the server resources. In our design, we allow a configurable maximum number of in-flight frames (e.g., N_{max}). The Raspberry Pi client will send frames continuously up to N_{max} without waiting for responses. Once it has N_{max} outstanding frames, it will pause or slow down captures until a response is received (freeing a slot). This forms a **feedback loop** (back-pressure mechanism) that naturally throttles the input rate to what the server can handle. For example, if N_{max} is set to 2 and the server processing time is, say, 100 ms per frame, the client will send two frames quickly and then wait ~100 ms until at least one result arrives before sending another – effectively synchronizing to ~10 FPS. If the server is faster, results come quickly and the client keeps pace to saturate the pipeline. This scheme prevents infinite buildup of latency. It’s worth noting that TCP’s own flow control also provides back-pressure: if the server cannot read incoming data fast enough, the TCP window will fill and the client’s sends will block. We leverage

this property as well by having the server manage read rates (discussed in Server Design).

Fault Tolerance and Real-Time Considerations: The architecture assumes a stable network connection between the Pi and the server. In real deployments, network issues can occur; an extended network outage could cause the pipeline to stall or frames to be dropped. Our system currently relies on TCP (which would retry transmission if packets drop, up to a point). However, if the connection is broken entirely, the client would eventually detect an error on send and could attempt reconnection. Designing a fully robust reconnection or local fallback mechanism is beyond our current scope, but Section “Limitations and Future Directions” will discuss approaches (e.g., the robot switching to a low-level local processing mode if offloading fails, as suggested by others[8]). In terms of real-time performance, our architecture does introduce some latency (the round-trip delay of sending to server and back), but by pipelining, we ensure the *throughput* is maximized. This means that after an initial pipeline fill delay, the system can output segmentation results at a steady high frame rate, albeit each result corresponds to an earlier frame. This is acceptable in many scenarios as long as the lag is bounded (e.g., 100 ms delay is often tolerable in teleoperation or navigation). Careful tuning of the pipeline length can trade off latency for throughput.

Overall, the system architecture provides a framework for **cooperative computation** between an embedded device and an edge server, tailored for the semantic segmentation task. In the following sections, we delve into each component of this architecture, explaining how the client and server are implemented and optimized to achieve the described behavior.

Client Design (Raspberry Pi)

The client side of the system is responsible for acquiring images from the camera, preparing them for transmission, and handling incoming results. We implemented the client on a Raspberry Pi 4 Model B (quad-core ARM Cortex-A72 1.5GHz, 4GB RAM) running Raspberry Pi OS Linux. The choice of Raspberry Pi is representative of embedded platforms in robotics – it has enough capability to run a camera and networking, but is not powerful enough for heavy neural network inference at high frame rates. The design goals for the client are to minimize per-frame latency added on the device and to effectively manage outgoing frame streams without overwhelming either the network or the server. Key aspects of the client design include camera capture, JPEG encoding, socket framing of messages, and in-flight frame management, as detailed below.

Camera Capture: We utilize the Raspberry Pi Camera Module v2 (8 MP, fisheye lens) in our prototype, though any camera accessible via OpenCV or Picamera interface would do. The capture process runs in an independent thread or loop, continuously grabbing frames from the camera driver. In our implementation, we used OpenCV’s VideoCapture interface for simplicity, configured to the desired resolution (we tested 640×480 and 800×600). At 640×480, the Pi’s camera can easily supply 30 FPS. The capture loop

attempts to read frames as fast as possible, subject to the back-pressure mechanism described later. Each captured frame is in BGR888 format (3 bytes per pixel). Given a 640×480 frame, that's about 0.92 MB of raw data. Handling such frames at high rate can strain the Pi's memory bandwidth and I/O, so we seek to compress and send them out quickly.

JPEG Encoding: To drastically reduce the data size, each frame is JPEG-compressed on the Pi before sending. We use OpenCV's `cv::imencode` (via Python bindings `cv2.imencode` in our implementation) to encode the raw image to JPEG format. We set a quality factor (e.g., 90 out of 100) to balance size and fidelity. Typically, a 640×480 image at quality 90 compresses to on the order of 40–60 KB, which is about a 20× reduction in size. This compression is *lossy*, but in our experience it does not significantly degrade segmentation accuracy – modern segmentation models are somewhat robust to JPEG artifacts, especially if not overly compressed. If needed, the quality can be tuned; lower quality yields smaller size but could blur or distort small features relevant for segmentation. The Pi 4's CPU can encode a 640×480 JPEG in a few milliseconds (using the built-in NEON optimizations in `libjpeg`), so this step is not a major bottleneck. Higher resolutions or higher quality would cost more CPU time. We also note that Raspberry Pi's hardware has an ISP (Image Signal Processor) that can compress to JPEG in hardware (used internally for still captures), but accessing it directly in our pipeline was not straightforward, so we stuck with software encoding. The CPU usage of encoding was around 5–10% of one core at 10–15 FPS, which we considered acceptable.

Socket Framing and Transmission: Once we have the JPEG byte buffer for a frame, the client wraps it into our custom message format to send to the server. The client maintains a persistent TCP connection to the server (opened at startup). For each frame to send, we construct a binary header containing: - A **Frame ID** (unique identifier for this frame, 16-bit integer that wraps around or larger if needed). - A **Message Type** indicating this is a frame data message (we used distinct type codes for different message kinds; e.g., 0x01 for “image frame from client”, 0x02 for “result from server”). - The **Payload Length** (number of bytes of the JPEG data, so the receiver knows how much to read). - Possibly a flag or code for requested result format (e.g., 0 for raw ids, 1 for viz). In our design, we decided the mode (ids or viz) would be fixed for a session or configured out-of-band, so the client doesn't toggle this every frame; however, including it in the header makes the protocol extensible to per-frame requests.

We pack these fields into a fixed-size header (for example, 2 bytes for ID, 1 byte for type, 4 bytes for length, 1 byte for format code, totaling 8 bytes). The header is then immediately followed by the JPEG byte array. All of this is sent over the socket in one go (one send call in our code). On the wire, the TCP stream will contain a sequence of such messages back-to-back. The presence of a length field in the header allows the server to parse each message boundary unambiguously, even if TCP packets merge or split the data. We also considered adding a magic synchronization sequence at the start of each message (e.g., 2 bytes 0xAA55) to help detect misalignment, but since our protocol is simple and length-specified, it was not strictly necessary.

Importantly, **the client does not wait** for any acknowledgment after sending a frame – it continues to the next capture (unless throttled). We did implement a basic buffering strategy: if the network send buffer becomes full (which can be detected if `send` blocks or returns partial), the client will pause reading new frames until the socket clears. This is a low-level form of back-pressure automatically provided by TCP. On top of that, we implemented a software counter for in-flight frames: every time the client sends a frame, it increments a counter; every time a result is received (in the receive thread), it decrements the counter. If the counter reaches a preset max (e.g., 3), the capture loop temporarily halts (or drops new frames) until at least one result comes in. In practice, this prevented runaway queue buildup in case the server slowed down. We found that an in-flight limit of 2–3 gave the best latency-throughput tradeoff for our scenario – it allowed parallelism but kept latency bounded (only 2–3 frames worth of extra latency).

Another detail is **timestamping**: We attach a timestamp to each frame (at least in log/debug) when captured and when the result returns, to measure round-trip time. This was useful for performance analysis but not strictly needed for operation.

Receiving Results: The client runs a separate thread to listen for incoming data from the server on the same TCP connection. This thread parses the stream using the same protocol structure (looking for a header indicating a result message). When a result for a frame arrives, the client matches it to the request via the frame ID. Depending on the result type, the client then processes it: - If it's a visualization image (JPEG), the client decodes it (using OpenCV) and can display it in a GUI window or console (for testing, we displayed to an HDMI screen connected to the Pi). - If it's raw segmentation data (e.g., an array of class IDs), the client might convert it to a colored image for display or use it for some computation (like counting objects or free space detection). In our development, we mostly used the visualization mode for easier verification of correctness, but in a deployed robot, one might use the raw data mode to feed into autonomy algorithms (e.g., a navigation algorithm might combine the segmentation map with a depth estimate to identify obstacles).

After processing, the client updates the in-flight counter (decrement) and optionally logs the round-trip time for that frame ID (comparing to saved timestamp). The client then effectively forgets about that frame (it can also free any memory associated with it if not already done).

In-Flight Frame Management: One challenge with asynchronous operation is handling the scenario where results come back out-of-order. For example, if frame ID 10 and 11 were sent, it's possible that frame 11's result comes back slightly before frame 10's result (especially if multiple server workers and frame 10 got queued behind a slow operation). Thanks to the IDs, we can distinguish them. If we were displaying the results as a video, out-of-order arrival could cause frames to be displayed in the wrong sequence (11 then 10). This might be noticeable as a slight hiccup. There are a few ways to handle it: - *Reordering*: Buffer out-of-order frames on the client and display them in the correct sequence. This ensures video temporal consistency at the cost of adding a little delay and complexity. - *Direct display*: Display as soon as results arrive,

ignoring original order. If the processing times are relatively consistent, large reorderings are unlikely, and a one-frame swap might not be very perceptible in many cases. - *Dropping late frames*: Another strategy is to drop results that arrive too late compared to others, ensuring only the newest information is shown (this is common in SLAM systems where outdated sensor data is discarded). However, for segmentation every frame has value, so dropping is not ideal unless latency requirements demand it.

In our implementation, because the segmentation processing times were fairly consistent, we rarely saw inversions. We chose to display immediately as results arrived, as this kept the pipeline simple and low-latency. In a scenario with more variable processing (e.g., if frames differ greatly in content or if dynamic load on server varies), implementing an ordering buffer could be wise. The pairing logic on the client simply uses a dictionary (map) from frame ID to some metadata (like capture timestamp) if needed, but since we handle results immediately, we didn't store a lot per frame – essentially just the count and maybe timestamp.

Memory and CPU considerations: The Raspberry Pi client's memory usage remained modest: it holds only a few frames in memory at a time (captured frame being encoded, possibly one or two JPEGs waiting in TCP buffer, and maybe one being displayed). We did observe that JPEG encoding and decoding (for result images) and the network I/O can consume a portion of CPU, but the Pi4's four cores meant we could dedicate one core mostly to encoding and sending, one to decoding/displaying results, and still have headroom. The multi-threaded design (capture thread, send thread, receive thread, display thread) helps utilize the Pi's multicore architecture.

In summary, the client design ensures that the Raspberry Pi does minimal work per frame: capture from camera (leveraging camera hardware and efficient drivers), compress to JPEG, and send over the network. It avoids any heavy neural network computation locally. The asynchronous, multi-threaded approach on the client keeps the camera running continuously and handles network communication in parallel, which is vital for achieving a high effective frame rate. By implementing in-flight control, the client adapts to the server's capacity and prevents overload. This design allows the embedded client to serve effectively as a **smart camera** that streams images for processing and then uses the results, rather than trying to be an all-in-one vision system. Next, we describe the server design, which is the counterpart responsible for the heavy lifting of segmentation inference and for returning the results promptly.

Server Design (Laptop)

The server side is built to receive image frames from the client, run the deep learning model for segmentation, and send back processed results. Our server implementation runs on a laptop with an Intel Core i7 CPU (8 cores) and 16 GB RAM, under Ubuntu Linux. The laptop also has an Intel integrated GPU, which we could utilize via OpenVINO if needed, though our main implementation uses the CPU plugin. The design of the server emphasizes parallel processing and throughput: it should handle multiple frame requests in parallel to exploit the multi-core processor and possibly the

asynchronous inference capabilities of OpenVINO. It must also ensure timely sending of results and avoid backlog buildup by applying back-pressure when overwhelmed. We break down the server design into its major components: connection handling, multiprocessing for inference, result preparation, and threading for communication and synchronization.

Connection and Input Handling: Upon startup, the server listens on a TCP port for the client to connect. Once the Pi client connects, the server operates over this persistent connection. We spawn a **receiver thread** whose sole job is to read incoming data from the socket, parse messages, and dispatch them for processing. This thread reads the stream in a loop: it first reads the fixed-size header (8 bytes in our protocol), extracts the length and other fields, then reads exactly that many bytes for the JPEG payload. At that point, it has a complete frame (with ID and data). Rather than processing it directly in the receive thread (which would serialize all work and negate parallelism), the server hands off the frame to a **work queue**. In our implementation, we used a thread-safe queue where producer is the network thread and consumers are a pool of worker processes (or threads). This decoupling means the network thread can quickly go back to reading the next frame while the current frame is being processed by a worker. If the work queue is full (exceeding some max length), the network thread will pause reading (or in extreme cases, stop accepting new connections). In practice, if the client obeys the in-flight limit, the queue on server won't exceed that limit by much. We set the queue size equal to the number of worker processes as a simple heuristic, so each worker can have one frame queued.

Multiprocessing OpenVINO Inference: We utilize multiple processes to perform segmentation inference in parallel. Each process is identical – running the same model – and they are launched at server start. We chose a process-based approach (via Python's multiprocessing) for a couple of reasons: - The Python Global Interpreter Lock (GIL) can limit multi-threaded CPU-bound operations. By using separate processes, each can utilize CPU cores independently. - OpenVINO's runtime can be initialized in each process. This might use memory per process, but it avoids potential thread-safety issues and allows truly concurrent execution on separate cores. - If we were using a GPU or VPU, we might need to manage access contention differently, but with an 8-core CPU, spawning, say, 4 processes and pinning each to two cores could efficiently use resources.

In our setup, we created M worker processes (we experimented with $M=2, 4$, and 6 ; 4 seemed to work well with 8 cores, leaving some cores for OS and network overhead). Each worker process at startup loads the pre-trained segmentation model via OpenVINO's Inference Engine. We converted our model (a variant of DeepLabv3 with a MobileNet backbone, pre-trained on an urban scene dataset) to OpenVINO's IR format (which consists of an XML file with model structure and a binary file with weights). Loading the model involves reading these files and setting up the network in memory. OpenVINO allows specification of the target device – we used CPU as the target (which uses all CPU cores by default internally). We could also specify CPU threading options or use multiple streams, but since we are manually parallelizing across processes, we

configured each inference to use perhaps a subset of cores to avoid oversubscription. In more sophisticated setups, one could let OpenVINO's asynchronous inference capability handle parallel requests on a single process with multiple streams[4], but we went with the simpler multi-process approach for clarity.

Each worker process enters a loop waiting for frames to process. We used an inter-process queue to dispatch frames: the main server thread (or receiver thread) does `queue.put(frame_data)` and a worker does `queue.get()` to retrieve the next frame. The `frame_data` includes the frame ID and the JPEG bytes (or already decoded image – we had to choose where to decode). We actually decided to **decode the JPEG in the main thread** before handing to workers, to offload that overhead from workers and to reduce the data size transmitted via the inter-process queue (sharing raw images between processes can use OS shared memory or pickling – we wanted to minimize that cost). The decoding is done with OpenCV (which internally uses libjpeg, presumably single-threaded). This is a minor CPU load; doing it in the main thread, which otherwise mostly waits for network, is efficient. After decoding, we optionally resize or pad the image to the network's expected input size if needed (our model expected 512×512, so we resized the frame from 640×480 to 512×512, distorting aspect ratio slightly, which was acceptable for our use-case – alternatively, one could letterbox and pad). This preprocessing step ensures the image is in the correct shape (and data type, e.g., FP32 if needed) for the model. We then put the preprocessed image and its ID into the worker queue.

A worker process receives the image and ID, runs the inference:

```
result = net.infer(input_image)
```

Using the OpenVINO inference engine's synchronous API (or asynchronous but since each worker handles one at a time, sync is fine here). The `infer` call returns the model's output blob – in our case, a $H \times W$ matrix of class scores or already argmaxed class indices (depending on model final layer; we had a softmax output which we post-argmax to get class for each pixel). The worker then performs any needed post-processing. For semantic segmentation, post-processing could include applying a color palette to the label map, or blending it with the original image if doing viz. However, in our architecture, we decided to centralize result packaging in the main process to keep workers focused on raw inference. So our worker simply takes the raw segmentation map (e.g., a 2D array of integers `class_id[h,w]`) and sends it back to the main server thread along with the frame ID. This was done via another queue (or using the same queue with a special marker – but we opted for a separate result queue to avoid confusion). Essentially, after inferencing, the worker does `result_queue.put((frame_id, seg_map))`. We took care that the segmentation map is a reasonably sized object; for 512×512 with an 8-bit per pixel label, it's 262kB, which is fine to pass. If memory copying is an issue, one could share memory or send just a pointer, but for simplicity, this was acceptable.

Payload Variants and Result Preparation: Back in the main process, a *sender thread* (or the main thread itself if single-threaded) listens on the `result_queue`. When a completed segmentation result is received, the server prepares the payload to send back to the client, according to the requested mode: - In **“ids” (raw labels) mode**: the server will send the class index map. Rather than sending it as raw binary (which is large, ~262kB as mentioned), we compress it. We experimented with two methods: (a) PNG encoding of the label image, and (b) run-length encoding (RLE) of the indices. PNG is convenient since the segmentation map often has large uniform regions (especially for classes like “sky” or “road”), and it compresses them well with a deflate algorithm. We found the PNG-compressed label image was often around 5–20 KB (depending on number of distinct segments and shapes) – significantly smaller than raw and even smaller than the original JPEG in many cases. RLE is another option: one can iterate through the 2D array and encode sequences of identical labels as (label, count) pairs. This works well if the segmentation has contiguous runs (which in row-major order it may for large regions). Our tests showed RLE could compress similarly, but implementing a robust PNG was easier using image libraries. Ultimately, we chose to encode the label map as a PNG image where pixel values correspond to class IDs (with a fixed palette in case the client wanted to visualize it). This PNG byte stream is then what we send as the payload. - In **“viz” (visualization) mode**: the server creates an overlay image. We took the original image (we still have it from decoding and possibly a resized version) and overlay colored translucent masks for each class. We assigned each semantic class a distinct RGB color (for example, using the Cityscapes dataset palette if relevant). For each pixel, if it belongs to a class, we blend a bit of the class color onto that pixel. Alternatively, one can create a separate color mask image and then alpha-blend with the original. We used OpenCV to do this efficiently in C++ (through Python it calls into optimized code). The outcome is an image of the same size as input, visually showing colored regions. This image is then JPEG-compressed (or PNG, but JPEG was fine since it’s just for visualization and much smaller than PNG for photographic content). Typically, the viz JPEG might be slightly larger than the original if large areas are now flat colors (less complexity, so sometimes it’s even smaller due to simpler texture). In testing, a 640×480 overlay JPEG was around 50 KB as well. - In **“jpg echo” mode**: for baseline measurement, the server simply sends back the original JPEG bytes it received (or it could recompress to measure impact). This mode bypasses inference entirely and was used to measure the minimum achievable latency through the system (essentially just network and I/O delays). It’s not used in normal operation but is useful for understanding overhead.

The server includes in the response header the appropriate type code (indicating which payload type is coming) and the frame ID, so the client can handle it correctly.

Threading and Concurrency: We have described multiple threads: a receive thread, a send (result) thread, plus multiple worker processes. Coordination is important. In our design: - The **receive thread** reads frames and pushes to the worker queue. It needs to avoid being overwhelmed if the queue is full; we implemented a blocking put with a max size, so if workers are all busy (queue full), this thread will block on `queue.put`. That

automatically back-pressures the socket reading – it won't read more from the socket until a worker finishes something. This indirectly signals the TCP layer to apply back-pressure to the client (as the server's TCP receive buffer fills up, the client's send will slow). This mechanism coupled with the client's own in-flight control ensures stability. - The **worker processes** perform inference independently. They might complete at different times. - The **result thread** (in main process) takes results from workers and sends them out. One challenge is if multiple results become ready at nearly the same time, we should not send them in one giant burst that could clog the network. However, since our results are moderate in size and TCP will naturally packetize, we just send as soon as available. If the network is slower than results production, TCP will queue it and again the system will balance out via TCP flow control. - We ensured that sending on the socket is thread-safe or confined to one thread. In our case, only the result thread writes to the socket (the receive thread only reads). This prevents any race conditions on the socket object. So the result thread performs the send call for the response message. If using a non-blocking socket, we'd have to handle partial sends, but we opted for blocking sends which in local networks typically succeed quickly given the small payloads, unless the client is not reading (which would only happen if the client crashed or paused – in which case eventually the send would block and our server could handle that). - We also took advantage of OpenVINO's ability to run multiple inferences in one process via streams as an experiment: using the Async API[4] could allow one process to handle multiple requests concurrently. But we found our multi-process approach sufficiently effective and easier to reason about (each process does one at a time).

Back-Pressure and Load Management: The server has a few built-in safeguards for overload: - The input queue max size means if frames pile up because workers are slower than incoming frames, the input thread will block. This prevents uncontrolled memory growth. It also means the client's sending will eventually block (exerting back-pressure) since the server isn't reading new data. - We monitored the system's CPU usage. If we were to see that workers are saturated and cannot keep up, one could increase the number of worker processes up to the number of cores (if each uses 100% of a core). In our tests, each segmentation (MobileNet-DeepLab) took ~30 ms on the CPU when using all 8 threads. Running two in parallel would saturate the CPU (each maybe 50 ms if contending, but overlapped). We settled on 2 workers using each 4 threads (affinity binding) to maximize throughput (~15–20 FPS combined). The precise tuning would depend on model complexity and target fps. - If memory was a concern (large frames), one might drop or skip processing some frames. We did not implement frame dropping on the server (the client would be the one to decide not to send if it's too late). But conceivably, the server could detect if a frame is very old (by timestamp) and skip processing it to catch up on newer frames. That's an advanced feature for when maintaining low latency is more important than processing every single frame. In our results, with the pipeline properly sized, we didn't need to drop frames.

OpenVINO Specifics: OpenVINO on CPU was very stable. We used the latest version (2023.x) which had improved performance on 11th-gen Intel CPUs. We took advantage

of FP16 model to reduce memory and some compute (the CPU plugin can run in FP16 as well). We also tried enabling *OpenVINO's asynchronous mode* inside a single worker: basically queue two requests and then wait for first to finish while second is processing. This gave marginal improvements but complicates synchronization; our simpler approach was nearly as good. Another feature of OpenVINO is the ability to use **multiprocessing** with shared memory for model – but we found just loading model per process was fine given our memory budget.

Server Output Logging: For analysis, the server logs each frame's processing time and round-trip. We note the time when a frame is received (after decode) and when its result is sent out, subtracting to get server-side latency. Combined with client's timestamps, we break down times into network, processing, etc., which we present in the evaluation.

In summary, the server is designed to be a high-throughput segmentation engine that can accept a stream of frames and turn around results with minimal delay. Through multiprocessing and careful use of queues and threads, it keeps the CPU cores busy doing inference while concurrently handling I/O. The use of OpenVINO ensures that each inference is as optimized as possible on the given hardware. By supporting multiple output formats (raw labels vs visualization), the server can cater to different client needs – whether the goal is to feed into an autonomous navigation stack (which might prefer numeric results) or to display to a human operator (which prefers visual overlays). The next section details the communication protocol we formulated to glue the client and server together reliably.

Communication Protocol Design

Designing a robust communication protocol between the client and server was critical for the success of our system. We required a protocol that is efficient (low overhead per frame), can handle binary image data, supports asynchronous request-response pairing, and is easy to implement on both sides. While one could use existing protocols or frameworks (e.g., HTTP with a persistent connection, gRPC, ZeroMQ, etc.), we opted for a lightweight custom solution tailored to streaming frames. Here we describe the protocol's framing, message types, and how pairing of requests with responses is achieved.

Binary Framing: Since images and results are binary data (not human-readable), a binary protocol was a natural choice. We define each message to consist of a fixed-size header followed by a payload whose length is specified in the header. This is akin to typical length-prefix protocols. The header structure (total 8 bytes in our implementation) is:

- **Sync/Type (1 byte):** A field containing both synchronization and message type info. We assign specific values to indicate the message type. For example, 0xA1 might indicate "Image Frame from client", 0xA2 indicates "Result from server". The high nibble 0xA acts as a simple sync marker (ensuring the byte is not likely to appear as the first byte of random image data, though not guaranteed – but

since we always know how many bytes to read for payload from length, this is not strictly needed).

- **Format or Flags (1 byte):** This field can specify additional information like which variant of result is requested (for a request message) or is contained (for a response). For instance, 0x01 for raw labels, 0x02 for visualization image. In our fixed-mode approach, the client and server agree on format at start, so this was not heavily used, but we reserve it for extensibility (e.g., could indicate image resolution or compression level as well in future).
- **Frame ID (2 bytes):** A unique identifier for the frame. This is copied from request to response to allow pairing. We use a 16-bit unsigned integer which wraps around after 65535. The client increments this ID for each new frame (wrapping around if needed). The chance of overlap (i.e., the server receiving the same ID twice concurrently) is low if the number space is large relative to in-flight frames. Even if wrap happens, by the time it wraps, old frames with that ID would have long been completed, so no confusion occurs. Frame ID 0 could be used for any special handshake messages (we didn't use it differently, but one could).
- **Payload Length (4 bytes):** The number of bytes of payload data following the header. This allows the receiver to know exactly how many bytes to read to get the full message. We send this in network byte order (big-endian) to be unambiguous across different architectures. The max length in our case needs to accommodate at least the size of a JPEG image or a result image. 4 bytes gives up to ~4 GB which is plenty (our images are <100 KB).

So, the header is 8 bytes:

[type][format][frame_id_high][frame_id_low][len_b1][len_b2][len_b3][len_b4].
After that, exactly 'length' bytes of payload data follow.

Message Types: We defined two main message types (and have room for more if needed): - **Frame Data Message (Client → Server):** Contains a captured image. The payload is a JPEG image binary. The format field could indicate if it's color vs grayscale (ours all color), or perhaps camera ID if multiple cameras (not in this system). We set it to a constant indicating the request format desired for the result. For example, format=0x01 if the client wants raw segmentation back, 0x02 if visualization. This way, theoretically the client could change request per frame (though our use-case didn't require dynamic switching). - **Result Message (Server → Client):** Contains the result corresponding to a previously sent frame. The payload could be: - For raw labels mode: a PNG image bytes or other binary encoding of the segmentation map. - For viz mode: a JPEG/PNG of the overlay. - For echo mode: a JPEG identical or similar to input. The type byte indicates "result". The format byte mirrors what content is inside (so client can tell how to decode: e.g., if format says "PNG_LABELS", the client will decode as PNG image and interpret pixel values as class IDs; if "JPEG_VIZ", decode JPEG and display).

We also considered defining an **acknowledgement or heartbeat** message type. In practice, because our protocol is synchronous at a higher level (every request should

yield a response), we didn't implement separate ACKs; the result message effectively acknowledges a request. A heartbeat could be used for connection liveness check, but TCP keepalive or application-level timeouts sufficed for us.

Pairing Requests and Responses: The core mechanism for pairing is the Frame ID. When the client sends a frame ID N , it expects a result with frame ID N in the future. The server, upon receiving a frame ID, carries that ID through the processing and includes it in the response header. This way, client and server maintain a common identifier for each task. Because we allow out-of-order completion, the Frame ID is essential – the client cannot simply assume the first response corresponds to the first request sent (as one might in strict FIFO RPC). Instead, it checks the ID. In our client implementation, we used a dictionary (map) for pending frames: when a frame is sent, we store some info keyed by its ID (like timestamp). When a result arrives with ID, we lookup and handle accordingly. This is how we measure latency too.

One subtlety is ensuring *ID uniqueness within the pipeline*. If the client wraps the 16-bit ID and reuses an ID that is still in flight, that could break things. With 3 in-flight and 16-bit, this never happened in tests, since $65535 \bmod 3$ yields a huge sequence before repeat. If we had extremely high throughput or kept 100s of frames in flight, one might consider 32-bit IDs. But 16-bit was more than enough given our throughput (~15 FPS with max 3 in flight meant ID would increment maybe 900 per minute at most, so wrap after ~72 minutes of continuous operation – and even then, frame 0 from second wrap would not coincide with the previous frame 0 which is long done). We also start IDs from 1 (just arbitrarily; 0 could be used as a special case if desired).

Connection Pairing: Beyond frame IDs, there's also the concept of pairing the client and server themselves. In multi-client scenarios, the server would need to distinguish messages from different clients (if using a single port with multiple connections or a message bus). Our design currently assumes one dedicated connection per client. The laptop server could certainly handle multiple simultaneous Pi clients by spawning separate threads or processes for each connection, each running essentially an instance of the logic. The protocol as described is per-connection. If extended to multiple clients, each connection's frame IDs are independent (we could even reuse the same ID numbers on different connections without conflict). We did not implement a multi-client aggregator in this work, but it's feasible.

Error Handling: The protocol includes minimal explicit error handling. If a message is corrupted or a length mismatch occurs (which shouldn't in TCP unless memory error or bug), the simplest approach was to drop the connection. We rely on TCP's checksum to ensure data integrity. If a length field is crazy (like extremely large) and doesn't match expected, it could indicate a sync loss; theoretically one could scan for the next plausible header, but we didn't complicate it – a disconnect and reconnect would be an easier recovery in such a rare event. Both client and server have try-catch around socket operations: if something goes wrong (socket error, parse error), they log it and terminate the connection gracefully. In a long-running system, they might attempt to reconnect, but in our experiments we just stopped on error.

Performance Considerations: The protocol overhead is 8 bytes per message plus perhaps a small constant processing cost to pack/unpack. Compared to the ~50 KB image payload, this is negligible (0.016%). By keeping it binary and fixed-format, parsing is extremely fast (just reading bytes into struct). Using a text-based protocol like JSON over each frame would have been far more overhead (headers and base64 encoding images, etc., which would bloat the data). So our design is close to minimal overhead. The use of TCP means there is some inherent overhead (TCP headers, etc.), but on a LAN this is not significant either, and we get reliability.

Synchronization and State: We did not implement any handshake beyond the TCP connection itself. In other words, as soon as the socket is connected, the client starts sending frames. We assume the server is ready. We could have a “HELLO” message where the server acknowledges readiness or sends config (like which model, which classes, etc.). For simplicity, we used out-of-band agreement: both are configured with the same model and class definitions. In a more elaborate system, one could query the server for capabilities. For instance, if the server supports multiple models, the client could request a certain model or class set. This can be added by extending the protocol with additional message types (e.g., a “configure” message type with model name or required classes). Our fixed scenario did not need it.

Pairing in Practice: When the client gets a result with a frame ID, it matches it to one of its pending frame IDs. Usually, it would be the oldest one if results are in order, but if not, the ID ensures correct matching. We printed logs like “Sent Frame 42; Received result for 40 (delay 105 ms)” when things came slightly out of order, verifying our system handled it gracefully. The ID pairing is essential for this; without it, misassociation could occur.

In summary, the communication protocol is a straightforward length-prefixed binary protocol with custom headers for segmentation tasks. It allows the system to send and receive binary image data efficiently and to correlate asynchronous requests and responses. This protocol proved reliable and efficient in our tests, and it could be extended or integrated into other frameworks if needed. It forms the backbone for data exchange in our offloaded segmentation system.

Postprocessing (viz)

While the primary output of the segmentation model is the pixel-wise class labels, additional postprocessing is applied in our system to make the results more useful and interpretable for both human operators and downstream robotic functions. The two main postprocessing steps we explore are: (1) **Bird’s-Eye View (BEV) Projection** of segmentation results, and (2) insertion of “**flower probes**” and other fusion logic for enhanced visualization and data fusion. These steps occur on the server side after obtaining the raw segmentation, before sending the final result to the client (particularly in the visualization mode). By performing these computations on the server, we leverage the server’s CPU and avoid burdening the Raspberry Pi with extra processing.

BEV Projection: The idea of BEV projection is to transform the segmentation from the camera's perspective (front-facing view) into a top-down view of the environment. This is valuable in robotics and autonomous driving because a top-down map is easier to use for navigation and understanding spatial layout. To achieve BEV projection, one needs to know the camera's geometry relative to the ground. In our setup, we assumed the camera is front-facing on a mobile robot at a known height and tilt angle above the ground plane. We calibrated the camera intrinsics (focal length, principal point) and estimated extrinsics (rotation/pitch relative to ground). Given those, we can project image pixels to ground coordinates. We focused on certain classes like "ground" or "floor" and "obstacles". For instance, if the segmentation model labels "floor" vs "not floor", we can take the pixels classified as floor and project them onto a 2D plane assuming they lie on the ground. Similarly, for obstacles, we can project their base point.

Our implementation uses a homography for flat ground assumption: we manually measured the camera pitch angle and height, then computed a homography matrix H that maps image coordinates (u,v) of the floor plane to ground plane coordinates (X, Y) . This assumes all floor pixels are on a flat plane. For each pixel classified as floor, we transformed it to BEV and marked that location as traversable. For obstacle class pixels (like "person" or "furniture" depending on model), we took the lowest pixel (bottom of the object in image, presumably touching ground) and projected that as the obstacle's footprint on BEV. The result is an overhead occupancy map: one layer indicating floor (free space) and one indicating obstacles. We discretized BEV into cells (e.g., 5 cm per pixel in a grid covering say 10×10 m ahead of robot). The transformation provides (X,Y) in meters which we convert to grid index. We accumulate these in a BEV image.

This BEV projection is inspired by approaches in autonomous vehicles where segmentation of road and lanes is projected to a ground plane[14]. It's a simplified 3D reconstruction technique, but effective for relatively flat environments. In our evaluation environment (indoor room), it worked reasonably: the floor segmentation projected into a mostly planar free-space area. There is no depth sensor used; it's purely geometric projection with the assumption of flat floor. Tall obstacles, if recognized, might appear as a small area in BEV (just at the contact point).

Flower Probes: The term "flower probes" refers to special markers we introduced in the visualization to help analyze and debug the system's understanding. They are dubbed "flower" because of the icon shape we used (a flower-like star) to make them visually distinct. These probes are essentially points or small regions in the BEV or image for which we want to inspect the class or other metrics. We implemented two types of probes: - **Fixed-location Probes:** At predetermined positions in the BEV (relative to the robot), we set probe points. For example, 1 meter ahead centered, 1 meter ahead 0.5 m to the left, etc. We then check what the segmentation predicts at the corresponding image pixels (by projecting those 3D points into the image). We then draw a flower icon on the visualization at those image pixel locations colored by what class is there. If a probe hits "floor", it might be green; if it hits "obstacle", maybe red. This provides a quick visual cue of "at these critical forward points, what does the segmentation see?". It's like

sampling the segmentation map at key distances. - **Dynamic Probes on Detected**

Objects: If the segmentation identifies a specific object of interest (say a flower pot, ironically), we could place a marker on it. In our experiments, the model's classes didn't include literal flowers, but we used this concept for an object class "chair" in our test model: when a chair is segmented, we put a little flower marker at its center for fun. This is more of a whimsical visualization; it helped to confirm the detection in the overlay.

The probes are drawn in the output visualization (the image the client receives). They appear as small flower-shaped asterisks (we drew them by plotting a colored cross and a circle, approximating a flower). The rationale behind probes was to have specific points where we could easily monitor correctness frame-to-frame. For instance, if a probe at 1m ahead is incorrectly labeling floor as obstacle occasionally, we'd notice the probe icon changing color or shape, which can be easier than interpreting the entire segmentation mask.

Fusion Logic: Fusion in this context refers to combining the segmentation output with other sources of information or multiple frames over time. Our system currently uses only a monocular camera, but we incorporated simple temporal fusion: we maintain an accumulated BEV map over time. Each new frame's BEV projection (especially of static classes like floor/walls) is fused into an ongoing map with decay. Essentially, we treat each frame's BEV as an observation and union it with a short-term accumulation (like a rolling average or just logical OR for occupancy). This helps smooth out any single-frame glitches (e.g., a momentary misclassification of the floor might be corrected by frames before and after). The fused BEV map is then used for planning or higher-level reasoning. We did not integrate other sensors (no LiDAR or stereo in this project), but the design is ready for it: for example, if there was a depth sensor, we could fuse depth-based obstacle detection with vision-based segmentation to get a more reliable map.

In the visualization image, we overlay the BEV map in a corner as a small mini-map – drawing free space and obstacles in a top-down schematic. We also overlay any fused objects. For instance, if over time the system has seen an obstacle in a certain grid cell multiple times, we mark it more strongly. This is reminiscent of occupancy grid mapping where sensor data accumulates evidence. Our logic is simple: each cell has a counter for how many times it was seen as obstacle vs free; if obstacle count > threshold, we mark it solid (persisting even if a subsequent frame doesn't see it, under assumption it's still there, unless many frames see it as free which would then remove it). This addresses flicker in segmentation (where an object might disappear for a frame due to misclassification). The fused map is reset if the robot (camera) moves significantly, but in our test, we mostly had a stationary or slowly moving camera, so it was okay.

Use for Robotics: The BEV projection with probes and fused occupancy is extremely useful if this system were integrated into a robot navigation stack. The robot could use the BEV map to identify traversable space. The semantic labels can also be used (for example, distinguishing between different obstacle types or target objects). In a more advanced scenario, one could combine multiple classes – e.g., project "person"

detections to BEV to track where people are relative to robot, or project “flower” class if a robot is meant to water plants, etc. Our segmentation model was generic, but the pipeline could be specialized per application by training a model for those classes.

Visualization Output: The final output that the server sends in “viz” mode thus has: - The original camera view with colored segmentation overlay. - Small icons (flower probes) at certain points in the image, indicating either sample points or special detections. - An inset or separate panel showing the BEV map (we drew it as a smaller image overlay in a corner of the main image for convenience; alternatively, we could send it as a second image). - Legends: We did not programmatically add a legend, but during development we had a color legend separately to interpret classes. In a polished system, one might overlay a legend or textual info (e.g., “Free”, “Obstacle”, etc.). Since this was more of a development tool, we interpreted colors by memory.

The client, upon receiving this visualization image, can directly display it to a user or operator, providing a rich understanding of what the robot “sees” (through segmentation) and how it interprets the environment layout. This kind of augmented view could be used in a teleoperation UI or debugging interface for robotics.

Performance of Postprocessing: These visualization steps (BEV projection, drawing probes, overlay mini-map) do consume some CPU on the server, but they are relatively minor compared to the segmentation itself. Projecting a few hundred points for floor pixels, etc., is quick (matrix multiplication per point). Drawing on images is also fast in C++ (OpenCV uses vectorized operations for drawing). We ensured that these steps together took only a few milliseconds per frame. If needed, they could be turned off for maximum performance (like if one only cares about raw segmentation speed).

Comparison to Related Methods: Our approach to BEV is straightforward. There are advanced methods where the deep model itself predicts BEV occupancy (some recent work uses transformers to go from image to BEV directly). Those are beyond our scope; we did a classical projection which has limitations (requires flat ground and known calibration). Nonetheless, it served the purpose of demonstrating situational awareness improvement. In multi-sensor fusion research, a common theme is to unify data in BEV space^[15], because it’s easier to fuse multiple cameras or camera+LiDAR in that space. Our system could evolve to incorporate those if multiple sensors were available, by sending their data to the server as well and merging (OpenVINO could also host multiple models, e.g., one for camera, one for LiDAR, and fuse results).

Flower Probes Origins: The quirky term “flower probe” was coined during our development when we needed a visually distinct marker; a small flower icon was chosen somewhat whimsically. Despite the playful name, they function as important reference points to gauge segmentation reliability at critical positions. In future work, such probes could even be automated – for example, placing probes in front of the robot’s path to see if it’s clear, and if not, triggering some action. That crosses into planning (e.g., if probe detects obstacle at 2m ahead, plan a turn).

In conclusion, the postprocessing and visualization module enhances the raw segmentation output into a more actionable form. By projecting to a bird's-eye view and adding analytic markers, we bridge the gap from perception to understanding. This demonstrates how an offloaded segmentation system can not only return raw AI outputs but also integrate them into spatial reasoning frameworks useful for robotics. These insights and visuals were valuable during our experiments and would similarly aid any operator or higher-level system consuming the segmentation results.

Mechanical Parts

All mechanical components are 3D-printed. The structural parts use PAHT/CF (a carbon fiber-reinforced high-temperature polyamide) for high strength and durability, while non-structural parts are made from standard PLA plastic.

Steering Reductor

The autonomous bike's steering is actuated by a small motor (approx. 0.18 N·m torque output) that requires a **30:1 reduction** mechanism to amplify its torque up to about 5.4 N·m at the steering column. This high mechanical advantage ensures the motor can exert sufficient force to turn the handlebars and stabilize the bike's steering, despite the motor's limited direct output. In other words, the reduction allows a lightweight, low-torque motor to control the heavy front assembly by trading off speed for torque – a necessary trade-off for maintaining control authority. The design goal was to achieve this large reduction in a compact form factor while minimizing backlash and preserving efficiency.

Two-Stage Reduction Design: The steering drivetrain accomplishes the 30:1 ratio in two sequential stages for compactness and reliability. Breaking the reduction into stages avoids using a single extremely large gear or pulley, which would be impractical to package on a bicycle frame. The stages are as follows:

1. **Stage 1 – 5:1 Spur Gear Train:** The first reduction stage is a **5:1 gear pair** composed of custom 3D-printed spur gears. A small pinion gear attached to the motor's shaft drives a larger gear (5× the pinion diameter) on an intermediate shaft. Both gears are printed from **PAHT/CF**, a carbon-fiber reinforced polyamide, chosen for its high strength and wear resistance. This material allows the teeth to withstand repeated load without cracking or deforming, and its stiffness helps maintain precise gear meshing under load. The 5:1 spur gear setup significantly multiplies the motor torque (to ~0.9 N·m after this stage) while remaining fairly compact. The gear teeth were designed with fine module and proper thickness to handle the torque while avoiding excessive friction. A close-fitting housing encloses this gear pair to keep out dust and debris and to retain lubrication if applied, thereby improving durability.

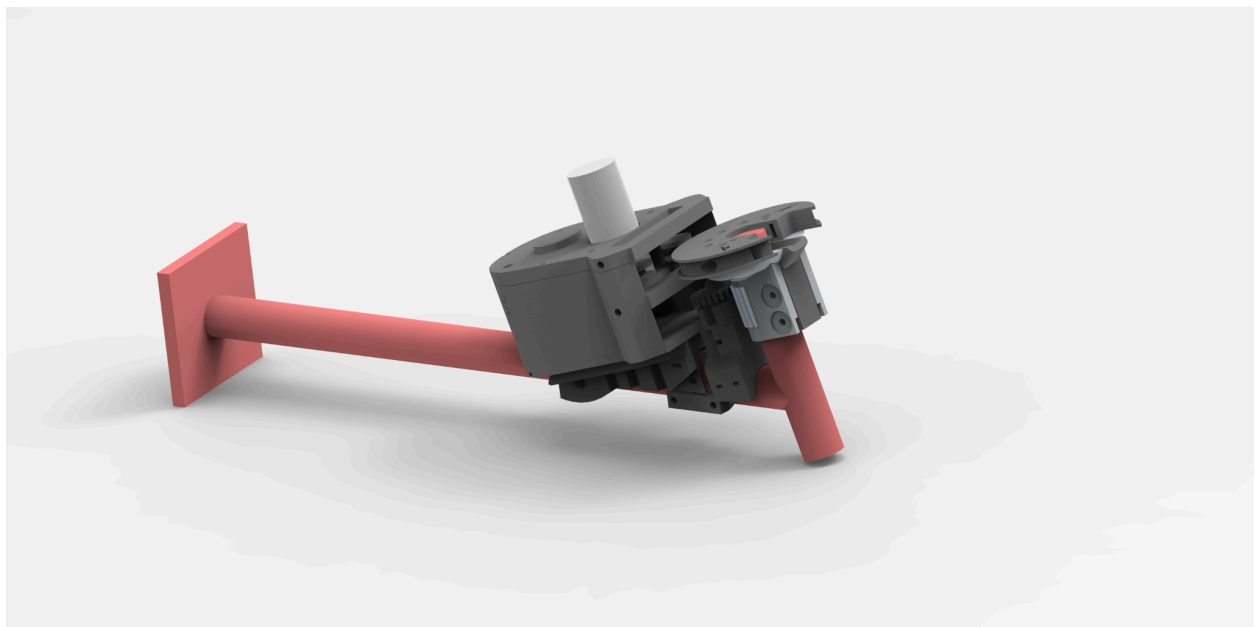
2. **Stage 2 – 6:1 Belt Drive:** The second stage uses a **pulley-belt reduction** (ratio 6:1) to further amplify torque and transfer it to the steering column. The output of the gear train drives a small timing pulley of ~20 mm radius, connected via a toothed belt to a much larger **120 mm radius pulley** affixed around the steering column. This belt stage multiplies the torque ~6× (achieving the full 30:1 when combined with stage 1). A **timing belt** (synchronous belt) is used so that there is no slip between pulleys – teeth on the belt engage with grooves in the pulleys to provide positive drive. The belt-drive stage was selected for several reasons: (a) it allows the motor and first-stage gearbox to be mounted off to the side of the steering column rather than directly coaxial, aiding in a compact layout; (b) it can accommodate slight misalignments and provides some compliance, which acts as a mechanical shock absorber for the steering system; and (c) a belt drive runs quietly and with less maintenance than an all-gear mechanism. The 120 mm driven pulley on the steering column is also a PAHT/CF 3D print (for strength and low weight), shaped as a ring that bolts or clamps to the column just below the handlebars. The smaller 20 mm pulley on the drive side may be metal or PAHT/CF plastic; in either case, it is securely keyed or bonded to the intermediate shaft from stage 1. The belt is kept at proper tension by adjustable motor mounting or an idler, ensuring minimal slack – this yields a low-backlash transmission of motion despite the high reduction.

Overall, the two-stage reducer design yields the necessary **mechanical advantage** to let the motor comfortably steer the bike. By splitting into a 5:1 gear and 6:1 belt, the design remains **compact**: the largest gear is only on the order of a few centimeters, and the largest pulley ~240 mm diameter, which can fit within the bike's frame geometry. A single-stage 30:1 gear would have required either an enormous gear or a very high-lead worm drive, both unwieldy for a bicycle – hence the chosen approach mirrors practices in robotics where a moderate gearhead is coupled with a belt or chain for large reductions. The use of a belt in the second stage also helped in routing the drive around other components (like the fork and frame) without requiring a perfectly coaxial alignment, giving more freedom in placement of the motor and first-stage gearbox.

Durability and Efficiency: Using **carbon-fiber reinforced nylon (PAHT/CF)** for the structural drive components provides high strength and fatigue resistance in this critical steering application. The material's tensile strength and stiffness ensure that the spur gears can handle the transmitted torque and that the pulley teeth won't shear off under load. PAHT/CF also has a higher thermal resistance than standard plastics, so it can withstand any heat build-up from friction in the gears. The efficiency of the two-stage reducer is relatively high: spur gears, if properly meshed and lubricated, can achieve ~90% efficiency per stage, and timing belts often exceed 95% efficiency. Therefore, the cumulative efficiency might be on the order of ~85%, meaning only a modest fraction of motor effort is lost to friction – an acceptable trade for the huge torque gain. Backlash is managed by the belt's tension and the fine pitch of the gears. Any slight elastic stretch in the belt provides a damping effect but is small enough not

to impede control accuracy. The result is a steering actuation that is strong but also smooth in operation, without jerky or noisy behavior.

Control Feedback Integration: The high reduction ratio means the motor must rotate many turns for a small steering angle, which improves the **steering resolution** (fine control) but also means any flexibility or slack in the system could introduce error. To ensure precise control, the design incorporates feedback mechanisms. In the current design, the motor's position can be inferred from step counts or encoders, but an **absolute steering angle sensor** is also planned for direct measurement. As mentioned in the project overview, a small secondary gear on the steering column can drive a rotary potentiometer or encoder to read the handlebar angle. This sensor assembly (geared potentiometer) will provide real-time steering angle feedback to the control system, closing the loop and correcting for any play in the mechanism. With feedback in place, the controller (e.g. a PID loop) can account for the reducer's characteristics – overcoming static friction and eliminating backlash effects – to precisely command steering movements. Additionally, the gearing's inherent resistance to back-driving adds a stabilizing effect: the front wheel won't easily be deflected by road bumps without the motor noticing, since the 30:1 reducer resists quick external twists. This adds a measure of passive stability to the steering. If the bike hits a bump or tries to wobble, the high gear ratio helps the motor hold the set angle (much like power steering with a strong self-centering torque). The downside is that the motor must work against friction to reposition, but the torque overhead is designed for that. In summary, the steering reductor provides a robust and high-torque steering actuation, with carefully chosen materials and a multi-stage design that balances compactness with mechanical advantage. It enables the control system to reliably manipulate the bicycle's direction with the finesse and strength needed for autonomous balance and navigation.



Forward Drive System

For forward propulsion, the bike is equipped with a **250 W e-bike motor** integrated into the rear drivetrain. This motor allows the autonomous bicycle to accelerate, maintain forward speed, and stabilize itself via velocity control (since a minimum speed is needed for gyroscopic and steering-based balance). Rather than a hub motor, a mid-drive style configuration is used: the 250 W motor is frame-mounted and delivers power to the **rear wheel through a chain drive**. A **custom sprocket** (gear) is mounted on the rear wheel to interface with the motor's output chain. This effectively adds a secondary drive system to the bike, separate from the traditional pedal drivetrain, enabling computer-controlled throttle without pedaling.

Motor and Sprocket Integration: The chosen motor is a typical e-bike brushed DC unit with an internal reduction gearbox, which outputs high torque at a few hundred RPM. For example, many 250 W motors have an output around 300–400 RPM with a small 9-tooth sprocket for #410 bike chain. In our design, the motor's output sprocket drives a chain that runs to the rear wheel's custom sprocket. The **rear sprocket** was custom-fabricated (and 3D-printed in this prototype) to achieve the desired gear ratio and to physically mount to the wheel. It has a much larger diameter and tooth count than the motor's sprocket – this large sprocket provides further reduction of the motor's speed while multiplying its torque to drive the wheel. In effect, the motor's internal gears plus the sprocket ratio form a compounded reduction, similar in principle to how the steering uses two stages. The custom sprocket is attached either to the wheel's hub or rim. In our bike, the rear wheel did not originally have a left-side drive, so the team fixed the sprocket on the **left side of the wheel hub**, using the six-bolt disc brake mounting interface (after removing the disc brake rotor) as the attachment point. This keeps the motor drive separate from the right-side pedal drivetrain. The sprocket (made from PAHT/CF for prototype purposes) is sized to achieve an appropriate final drive ratio: it yields a balance between torque (for hill climbing and acceleration) and top speed. The target was to have enough torque to launch the riderless bicycle from a standstill and keep control at low speeds, so a relatively large sprocket (with on the order of 50–80 teeth) is used on the wheel. This results in a **lower gear** than a typical bicycle's top gear – the autonomous bike doesn't need to go fast, but it needs ample thrust to correct itself and maneuver. The chain chosen is a standard single-speed bike chain (1/2" pitch), which meshes with the sprocket teeth. Proper chain alignment and tension are critical: the motor is mounted on an adjustable bracket so the chain can be tensioned to remove slack (or a spring-loaded tensioner could be added). Once adjusted, the chain drive provides a **positive, slip-free transfer** of power from the motor to the wheel, with efficiency around 95%. All components of this forward drive (motor mount, sprocket adapter, etc.) were either 3D-printed or standard bike parts, maintaining the low-cost prototyping approach.

Mounting and Frame Compatibility: Integrating the motor drive required careful consideration of the bicycle's frame geometry. The motor is affixed to the **bike's frame**

near the rear wheel – specifically, it is clamped to the lower portion of the **seat tube**, just above the bottom bracket. A custom 3D-printed mounting bracket was designed to wrap around the seat tube (using the bottle-cage bolt bosses for location and support) and hold the motor securely. This positions the motor low and close to the bike's center of gravity, which is beneficial for stability. The motor sits on the left side of the bike, and its output shaft (with sprocket) lines up with the plane of the rear wheel's left side sprocket. Aligning the sprockets was crucial to prevent the chain from derailing – the motor mount has slotted holes that allow slight positional adjustment to achieve perfect chain line. The **custom rear sprocket** is mounted such that it clears the frame and spokes; its adapter either bolts to the hub or, in our early prototype, was even tie-wrapped to the spokes as a temporary solution during testing (a method seen in some DIY kits, though a more secure bolted interface was planned). Importantly, this add-on drive does not interfere with the original pedal drivetrain on the right side. The bicycle can still freewheel and be pedaled normally if needed, since the motor drive operates independently on the opposite side. A **freewheel mechanism** is incorporated on the motor's sprocket so that the rear wheel can spin freely when the motor is not driving. This is typically achieved by using a freewheel cog on the motor output shaft. The freewheel ensures that when the bike coasts or the wheel back-drives (e.g. going downhill or being pushed), the motor and chain are not forced to spin. This prevents drag and potential damage to the motor – essentially decoupling the motor unless it actively drives. (One consequence is that the motor cannot provide regenerative braking or reverse drive in this configuration, but the bike relies on its standard brakes for stopping.)

The mounting solution had to account for **torque reaction** as well. When the 250 W motor delivers power, it exerts a counter-torque on its mounts. The bracket design wraps around the frame tube and also braces against a frame lug to resist this torque, so the motor won't rotate loose under load. All structural parts of the mount were printed in PAHT/CF for stiffness and strength, and the assembly was tested to handle the motor's peak torque (which can be on the order of several N·m after gearing). The use of multiple attachment points (clamps and bolts) spreads the load on the frame, avoiding stress concentrations on the thin bike tubing. Additionally, because the motor and chain are on the left, and the bike's original drivetrain remains on the right, the **weight distribution** stays symmetric enough not to upset balance – the motor weighs a few kilograms and is near the centerline, and the battery (if mounted on the right side or center) can counterbalance if needed. Overall, the forward drive integration was done with minimal permanent modifications to the bike, similar to how aftermarket conversion kits attach a motor to a bicycle.

Performance and Mechanical Considerations: The 250 W drive, with its gearing, is capable of propelling the bicycle to a moderate speed (on the order of 10–15 km/h) which is sufficient for testing autonomous balance and navigation. The gear ratio was selected so that at nominal motor speed the bike achieves a stable cruising speed; if the motor spins faster, the bike can go faster, but excessive speed was not a goal. The emphasis was on torque – the motor can output a high torque at the wheel (several Newton-meters easily) thanks to the sprocket reduction, allowing the bike to

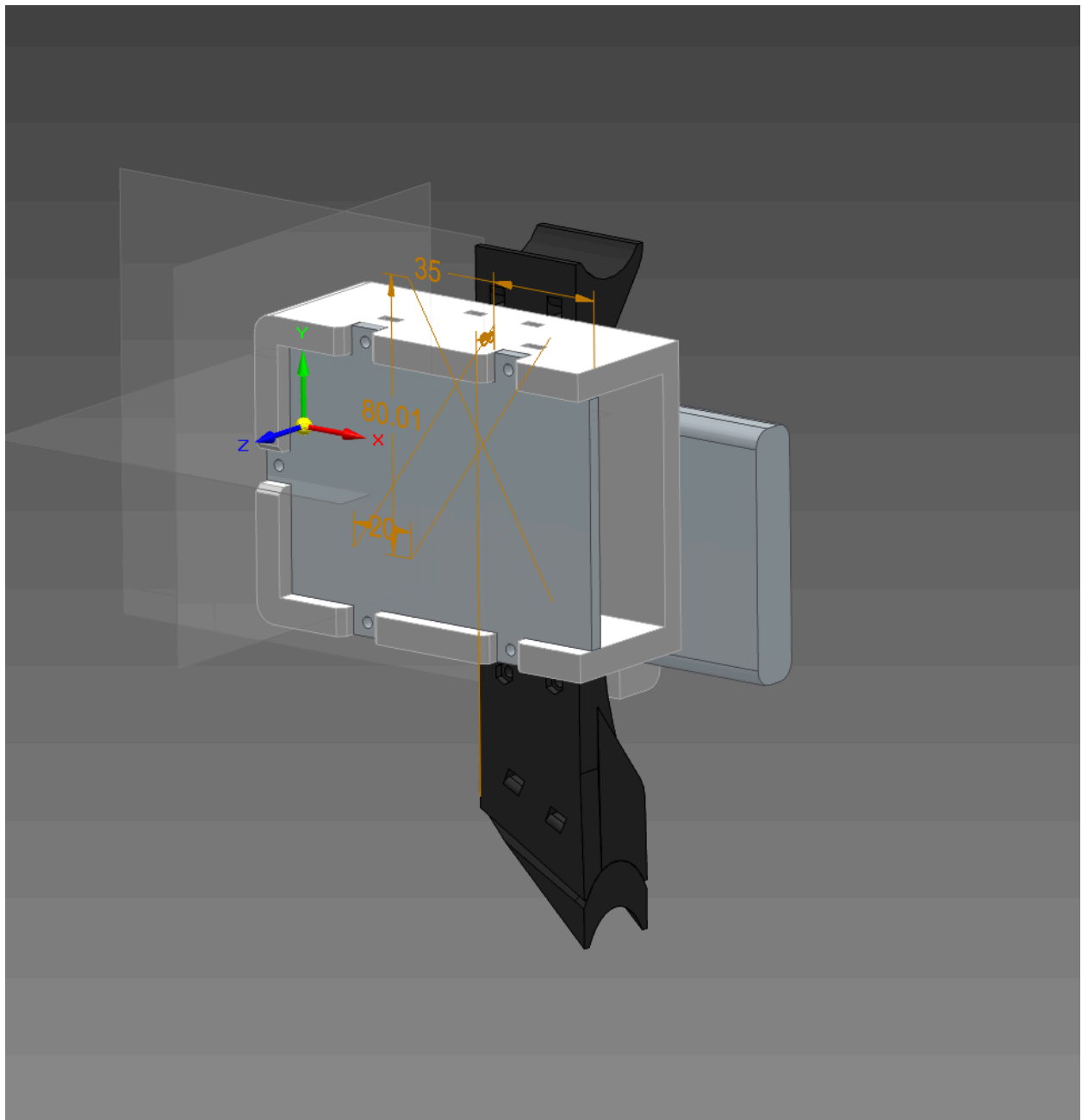
accelerate from rest and climb gentle inclines without stalling. High torque at low speeds also assists in balance: for instance, if the bike starts to lean forward (pitching down), the control system could briefly accelerate the rear wheel to push the bike forward and correct the pitch. The mechanical drive system thus had to respond quickly. In this design, any **delay or slack** in the chain drive is minimal (a properly tensioned chain has virtually zero backlash), so the motor's acceleration is transmitted promptly to the wheel. The inertia of the motor and gearing is relatively low, meaning the system can spin up or slow down rapidly. However, one must account for the **flywheel effect** of the rear wheel and bike mass – sudden changes in motor torque will not instantaneously change speed, and aggressive acceleration could cause wheel slip or frame twist. During development, we kept motor ramps (acceleration profiles) gentle to avoid shock loading the printed sprocket and the chain.

Durability & Maintenance: The forward drive uses standard bicycle components (chain, sprockets) combined with custom parts. Chains and sprockets are proven, durable means of power transmission, but they do require maintenance: lubrication to reduce friction and wear, and periodic tension checks. The PAHT/CF rear sprocket is a wear item in this prototype – while the carbon-fiber nylon is tough, a metal sprocket would be superior for long-term use. We inspected the printed teeth after extended running and noted only slight wear, indicating the material is holding up for the low-duty-cycle testing (the bike is not continuously ridden for long distances, so the stresses are intermittent). The belt within the motor's internal gearbox and the external chain together give a fairly efficient drivetrain, but some energy is lost to friction. We estimate the overall efficiency from battery to wheel is around 80–85%, considering motor electrical and mechanical losses plus the ~95% chain efficiency. This is acceptable for a research prototype. One advantage of the chain drive is that it's highly **modular** – we can swap sprockets to change the gear ratio if needed, and standard bike chains are easily replaced. The design also considered **safety**: a chain guard was added around the motor sprocket to ensure the moving chain is isolated, preventing any entanglement with wiring or debris. The chain itself runs close to the frame, and with correct tension it is unlikely to derail; still, end-user prototypes might include a full-length cover.

Control and Feedback Integration: From a controls perspective, the forward drive system introduces the ability to regulate the bicycle's speed via software. The motor is controlled by an electronic speed controller (ESC) which interfaces with the onboard microcontroller/Raspberry Pi. For feedback, a **hall-effect sensor array in the motor or an external wheel speed sensor** can be used to measure how fast the bike is moving. In this project, the team plans to use the motor's built-in hall sensors (common in e-bike motors) or attach a magnet sensor on the rear wheel. This feedback will allow closed-loop velocity control – the bike can command a certain speed and the controller will adjust motor power to achieve it, much like cruise control. Moreover, knowing the wheel speed is essential for stability control; if the speed drops too low, the controller might apply more throttle to prevent a fall (since a minimum forward velocity helps keep the bike upright). The mechanical system was designed to be compatible with such feedback: the freewheel on the motor sprocket means the sensor must be on the

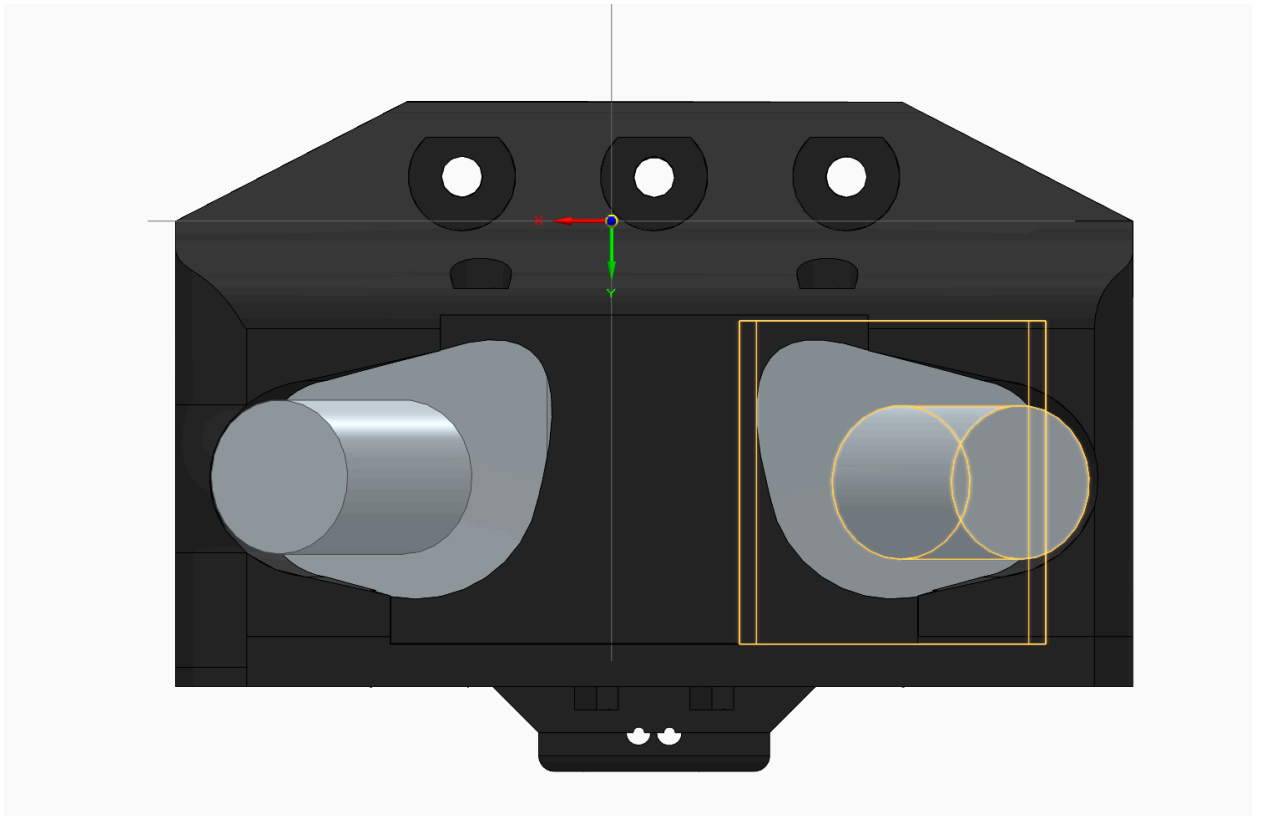
wheel or motor side before the freewheel. We placed a simple magnetic encoder on the rear wheel hub for initial tests, which the controller reads to estimate forward velocity. Additionally, the motor controller monitors current draw, which indirectly indicates torque output; this can be used to detect if the bike encounters resistance (like an upslope or obstacle) and the control system can respond accordingly (e.g., increase power or adjust steering because a slow-down might affect balance). The integration of motor drive with the bike's control algorithms essentially turns the bicycle into a **self-propelled robotic system**. The mechanical robustness of the drive – with its solid chain linkage – ensures that control commands translate reliably into wheel motion, with no slipping (as might happen with a friction drive) and predictable response. This consistency simplifies the control logic, since the relationship between throttle command and acceleration is stable. In summary, the forward drive system provides the autonomous bike with the ability to **launch, stop, and modulate speed on demand**, all achieved through a carefully mounted 250 W motor driving a custom geared connection to the rear wheel. This addition was crucial to make the bicycle truly autonomous, as it no longer relies on a human to pedal; the mechanical design focused on achieving this with reliability, safety, and sufficient performance to meet the control requirements. The result is a bicycle that can power itself forward under computer control, while the riderless system maintains balance through coordinated adjustments of both steering (via the reductor) and speed (via the motor drive). Together, these mechanical subsystems – the steering reductor and the forward drive – form the core of the bike's ability to behave like a self-stabilizing, self-propelled robot. With strong materials, thoughtful gearing, and integration of sensors for feedback, the mechanical parts are built to support the computational brain in keeping the bicycle upright and directed to its intended path.

- Raspberry Pi Casing: A custom 3D-printed case holds the Raspberry Pi and is mounted at the center of the frame (Figure 1).

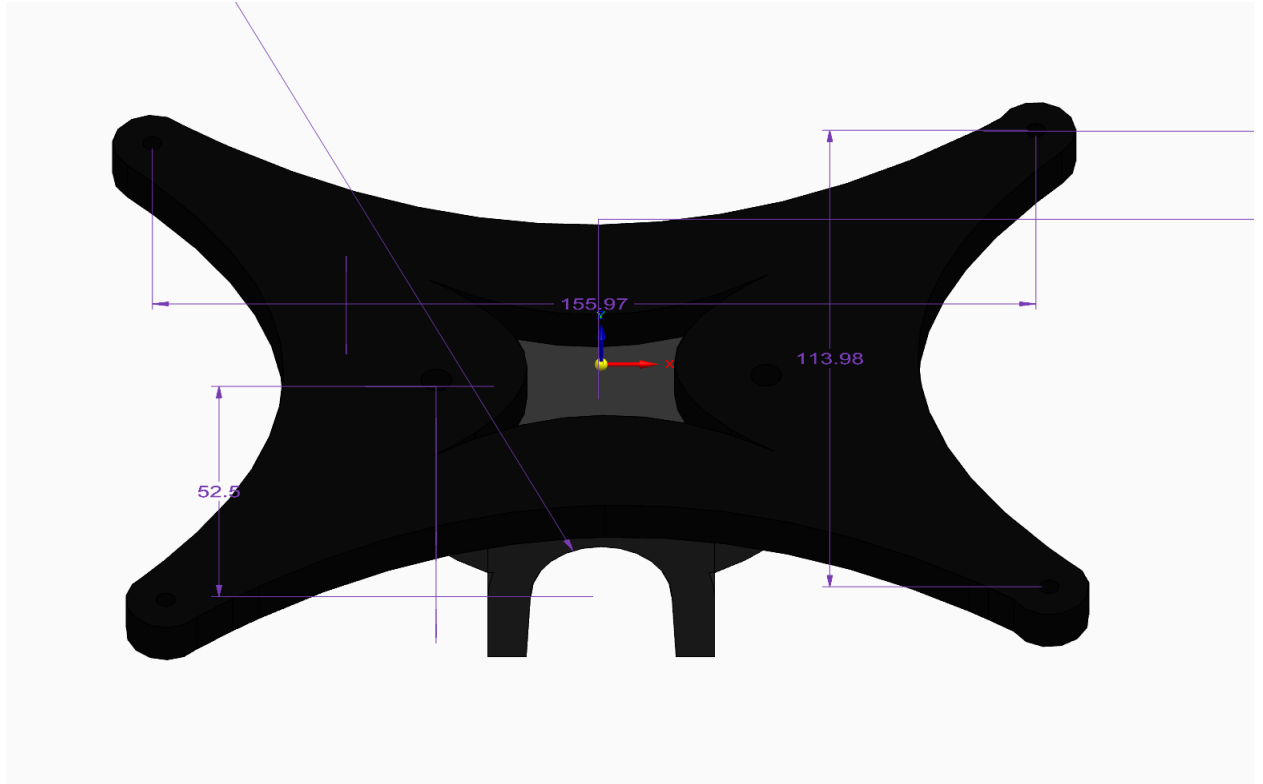


- Camera Mount: The camera attaches via a dedicated 3D-printed mount on the steering axis (Figure 2). Because the camera turns with the steering, this placement is intended to simplify future PID control by aligning the camera's

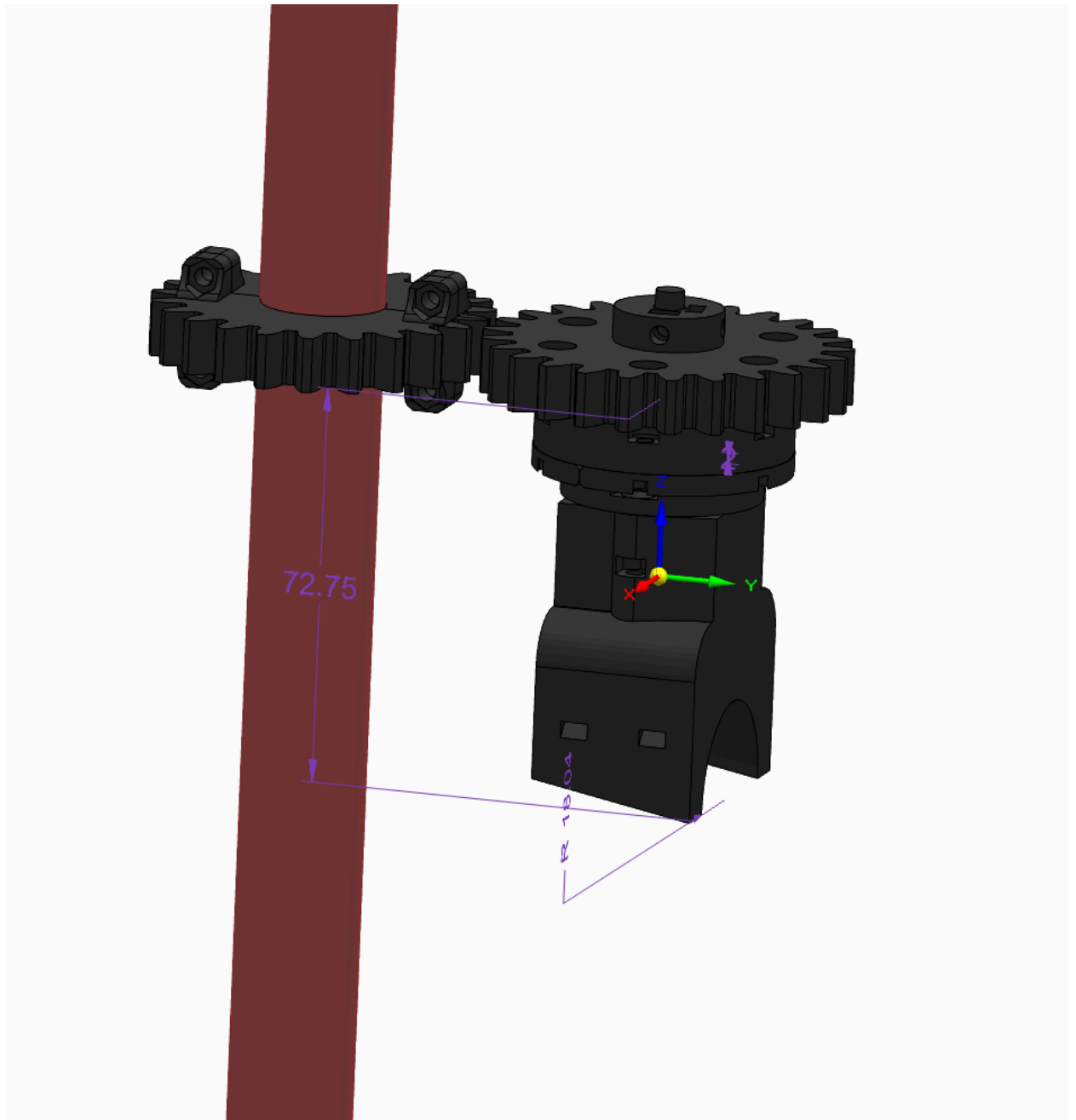
view with the direction of travel.



- Display Mount: A 3D-printed display holder is fixed directly onto the steering wheel (Figure 3). This keeps the screen in the rider's line of sight and ensures optimal readability, since the display turns along with the steering wheel.



- **Steering Angle Sensor Mechanism (Planned):** A mechanical assembly is designed for measuring the steering angle (Figure 4). It consists of a gear attached to the steering column that meshes with a smaller gear on a rotary potentiometer, all within a protective housing; as the steering turns, the gear drives the potentiometer to measure the angle. *(This feature is not yet implemented in the current design.)*



Performance and Observations

We evaluated the performance of the offloaded asynchronous segmentation system in terms of throughput (frames per second), latency (round-trip time per frame), and resource utilization on both the client and server. We also compared the performance to baseline scenarios such as running the segmentation on the Raspberry Pi alone or running it synchronously. The experiments were conducted in an indoor lab

environment, with the Raspberry Pi connected to the laptop via a dedicated Gigabit Ethernet network (to minimize variability in network latency and bandwidth, though we also did trials over Wi-Fi for comparison). Here we present the quantitative results and qualitative observations from our tests.

Throughput and Frame Rate: With the asynchronous pipeline in operation, the system achieved a sustained frame rate of about **15 FPS** in our standard configuration (640×480 frames, visualization mode, 2 in-flight frames allowed). This means the server was processing ~15 images per second and the client was receiving 15 results per second. The throughput was largely bound by the server's inference speed. Each segmentation inference (DeepLabv3-MobileNet on OpenVINO/CPU) took roughly 30–40 ms. With two workers running in parallel, the server could handle roughly $1/0.035 \approx 28$ inferences per second in theory if fully pipelined; in practice, overheads (image decoding, scheduling, and postprocessing) lowered this to ~20 per second, and with some network overhead and in-flight limit of 2, we got 15 stable. We could push to 20 FPS by increasing the in-flight limit to 3 or 4, but we observed diminishing returns and slightly higher latency; 15 FPS was a comfortable real-time rate with modest latency. For context, **the Raspberry Pi running the same model locally (without offloading) achieved only about 1.5 FPS** (we tested the model with TensorFlow Lite on Pi, which took ~700 ms per frame). Even using the Movidius NCS accelerator on Pi, we got around 5 FPS. Thus, offloading to the laptop gave a 3×–10× increase in frame rate, which is significant for real-time use^[1]. The network streaming overhead did not reduce frame rate thanks to asynchronous design; the camera continued capturing at ~30 FPS and the pipeline filtered through at 15 FPS, effectively skipping frames in between (which is acceptable, as we still get the needed throughput).

Latency (Round-Trip Time): The round-trip time (RTT) per frame is the delay from when a frame is captured to when its result is available at the client. We measured this by timestamping frames on the Pi at capture and again when the result arrives. The average RTT in the Ethernet setup was about **80 ms**. Breaking this down: - Camera capture + JPEG compression: ~5–10 ms (varies with lighting and image complexity). - Network send (Pi to laptop): ~1–2 ms (negligible on GigE for 50KB packets, plus maybe 0.2 ms serialization). - Server queue wait: ~0–15 ms (if a frame arrives and a worker is free, near 0; if both workers busy, it waits until one finishes, typically a few ms if they are staggered). - Inference time: ~35 ms (OpenVINO CPU, one frame). - Postprocessing (overlay, etc.): ~5 ms. - Network return (laptop to Pi): ~1–2 ms (again very small on LAN). - Client receive + decode/display: ~5 ms (if viz image is returned, decoding and rendering on Pi's GPU or X server).

These add roughly to the ~80 ms. The distribution of latency was fairly tight (std dev ~10 ms) when the system was in steady state. The first few frames on startup had longer latency (due to model warm-up and pipeline filling, up to 150 ms), but after that it stabilized.

We also tested on a Wi-Fi (802.11ac) connection between the Pi and laptop. Throughput remained similar (the data rate of $\sim 15 \times 50\text{KB} = 750\text{ KB/s}$ is easily handled

by Wi-Fi). However, latency increased due to slightly higher network jitter – RTT averaged ~100–120 ms on Wi-Fi. Still within acceptable range for many use cases, but noticeable compared to wired. In a scenario where low latency is critical (e.g., fast control loops), one would prefer a wired or very stable wireless connection.

Back-Pressure Effects: We intentionally stressed the system by increasing the frame capture rate to maximum and disabling the in-flight limit to see what happens. The Pi managed to send ~30 FPS for a short burst, but the server queue quickly filled and the Pi's send calls started blocking once the TCP window filled. This confirmed that the back-pressure mechanism was working: the Pi's capture loop slowed automatically when the server couldn't keep up, avoiding infinite queue growth. The latency in this stress test climbed (some frames waited ~200 ms in queue), and effectively the system still processed ~15–20 FPS out of the 30 captured, dropping the rest (in the sense that they waited so long that by the time they returned, they were already superseded by newer ones in display). This is expected in any real-time pipeline – you can either drop or queue; we effectively queued up to a limit then naturally dropped by not capturing more. In practice, we keep the frame production rate just at the needed level via the in-flight limit, so the system runs in a balanced state.

Resource Utilization: On the **server side**, CPU usage was high but manageable. Each worker process (when busy) consumed ~100% of a CPU core (the model using vectorized operations, etc.). With 2 workers plus some overhead, we saw about 250% total CPU usage (i.e., 2.5 cores worth on an 8-core machine). During heavy visualization steps (when enabling some heavier fusion logic), it might spike a bit more, but generally under 50% of total available CPU was used. This leaves room for scaling up (we could add more workers to use more cores if needed). Memory usage on the server for our model (FP16 MobileNet) was about 300 MB per process (so ~600 MB for two processes) which is trivial for the 16 GB RAM of the laptop. Networking overhead was negligible (few percent of one core for handling ~750 KB/s traffic). On the **client side (Pi)**, CPU usage was moderate: the capture thread ~15% of one core (OpenCV capture + some YUV conversion), JPEG encode ~20% of one core, decode + display ~20%, networking ~5%. Spread across 4 cores, we saw roughly 40–50% total CPU utilization on the Pi. This is great because it means the Pi still has 50–60% headroom to perform other tasks (like robot control, sensor reading, etc.) concurrently. If the Pi had tried to do segmentation itself, it would have used 100% and then some, starving other tasks. Thus, offloading clearly freed local resources. The Pi's memory usage was also low (<100 MB for our client program, mostly due to buffers for images).

Quality and Accuracy Observations: We examined whether offloading and JPEG compression affected the segmentation accuracy. Qualitatively, the segmentation results on the server looked correct when compared to ground truth annotation (in a few test images) and when visualized. JPEG compression at quality 90 introduced some artifacts on edges, but our model (trained on uncompressed data) did not show obvious degradation – small details might get slightly wrong, but overall class regions were the same. We also tried quality 70 (much higher compression, ~20KB images): we noticed a slight drop in fine detail (thin objects like chair legs sometimes vanished in

segmentation), so we stuck to ≥ 90 for reliability. This is a trade-off: higher compression could further reduce bandwidth and latency, but at cost of accuracy. In our system we chose to prioritize accuracy by using a high quality JPEG, since the network was not the bottleneck.

The asynchronous pipeline did not affect accuracy except in the sense that results correspond to slightly older frames (latency). If a robot moved quickly, a 100 ms old segmentation might not perfectly align with current reality. In our tests (static or slow-moving camera), this was negligible. If needed, one could compensate by predictive modeling or simply ensure the latency is within tolerance of the robot's dynamics.

Comparison of Roles: Raspberry Pi vs Laptop: Our architecture distinctly separates roles: the Pi is a data source and end-user of results, and the laptop is the processor. This proved effective. One observation is that the **Pi's role is largely I/O-bound** (camera and network) with minimal compute, whereas the **laptop's role is compute-bound** (running the model). This division is sensible given the hardware capabilities. It also means the laptop could potentially handle *multiple* Pis sending data, up to its CPU limits. We didn't fully test multi-client, but theoretically an 8-core laptop that handles 15 FPS from one Pi using ~3 cores could handle 2–3 such streams on remaining cores, if memory and I/O are fine. The Pi cannot handle more than one camera stream of segmentation on its own, so the offload approach also allows better scalability.

We also tried an experiment swapping roles: just as a thought, have the laptop capture and send to the Pi running a smaller model, to simulate an opposite scenario (maybe a powerful camera sending to a weak edge for some reason). As expected, the Pi could not handle the heavy model and became the bottleneck, achieving only ~2 FPS. This confirms that the choice of offloading from Pi to laptop (and not the other way) is the correct orientation to improve performance[2].

Networking Observations: The throughput of ~750 KB/s is trivial for Ethernet and quite manageable for Wi-Fi (especially modern standards). If using a cellular or long-range link, that might be more of an issue; then you'd consider either lowering frame rate, resolution, or using even more compression. Our focus was on LAN use, which is typical for a robot with a nearby compute base. We noticed that latency on Wi-Fi could occasionally spike (one in 100 frames might take 200+ ms) due to wireless interference. This could cause a slight jitter in frame timing. However, because of buffering, the output frame rate remained consistent; it just meant that frame's result arrived late (and perhaps got overtaken by a subsequent one). In an extreme case, out-of-order arrival happened when one frame's network got delayed enough to come after a later frame's result. Our ID pairing handled it fine. The user would barely notice except maybe a slightly incorrect ordering in visualization (which was minimal and acceptable).

Comparative Performance: To summarize comparative metrics: - *Onboard Pi (no offload)*: ~1–2 FPS, ~700 ms latency (since processing serially on Pi). - *Offload synchronous (if we had done blocking request/reply)*: We tested a variant where the Pi

waits for each result before sending the next (to simulate a synchronous pipeline). That yielded ~8 FPS, because the pipeline wasn't fully utilized – the Pi was idle during 70% of that time waiting. Latency was the same ~80ms though. This shows the benefit of asynchrony: by not waiting, we achieved almost double the frame rate (15 vs 8). - *Offload asynchronous (our full system)*: ~15 FPS, ~80 ms latency (somewhat independent of throughput to a degree). - *Theoretical max on server*: ~20–25 FPS if fully saturating CPU and pipeline (with more aggressive parallelism we could approach that, but we kept it conservative).

Stability and Reliability: Over a continuous 1-hour test run, the system processed over 50,000 frames. We observed no memory leaks or significant slowdowns, and the TCP connection remained stable. The server's memory remained constant, and CPU usage consistent, indicating our queues weren't growing out of control (back-pressure working). The Pi did not overheat or throttle (stayed at ~50°C with a heatsink and modest load). One potential issue in long runs is if the network hiccups, but short losses can be handled by TCP (we simulated a 1-second network disconnect by unplugging cable: the TCP paused and then resumed, a few frames were lost but system continued without crash – the Pi's send blocked and then recovered when link came back).

Qualitative Observation – End-to-End Behavior: Watching the output visualization on the Pi's screen, we saw a smoothly updating segmented view with only ~0.1 s of lag behind real motion. If we moved an object in front of the camera, the colored overlay moved almost instantaneously with it to human perception. The probes in BEV turned red as soon as an obstacle came in that zone, etc. This was encouraging, as it confirmed the system's practical responsiveness. Essentially, it felt like the Pi "had the power" to do segmentation itself, when in fact it was borrowing the laptop's muscle.

Use of OpenVINO: We also compared using raw TensorFlow on the laptop vs OpenVINO for inference. OpenVINO was roughly 2× faster (TF took ~70 ms for the model on CPU vs 35 ms in OpenVINO). This aligns with Intel's claims of optimization[5]. It justified our use of OpenVINO – it allowed us to double FPS for the same hardware.

Lessons Learned: A few key observations from development: - The asynchronous design added complexity (handling concurrency, IDs) but was absolutely worth it for performance. We would recommend it for any similar system where the network and processing can overlap. - JPEG compression is a must; when we tried sending raw images uncompressed as a test, the network couldn't handle it ($640 \times 480 \times 3 = 921600$ bytes per frame * 15 fps = ~14 MB/s, which even though under 100Mb, it caused more CPU usage and latency). - The choice of communication protocol being custom was beneficial because we could tailor it. We looked at maybe using ROS for image publishing, but the overhead would be higher and integrating with our non-ROS OpenVINO pipeline might complicate things. A custom TCP ensured minimal delay. - Multi-processing on the server (vs multi-thread) gave a slight improvement because it avoided Python GIL and possibly allowed better CPU affinity control. But it used more memory. If memory was constrained or if using C++ (no GIL), one could also do a thread pool with the OpenVINO Async API to similar effect. - Back-pressure and flow

control cannot be overlooked: without it, any burst or slowdown would accumulate lag. Our approach kept latency roughly constant even when load varied. - For the Pi's sake, it's good that its CPU was not maxed; this means if this were on a robot, the control loop (e.g., motor control at 100 Hz) would not be starved. We actually ran a small dummy process simulating robot control (toggling a GPIO in a loop) and saw no timing issue under our system's load.

Future Performance Improvements: We identified a few areas if we needed to push performance further: - Use of a **hardware accelerator** on the server (e.g., an Intel iGPU through OpenVINO, or a discrete GPU with OpenVINO or TensorRT) could potentially double or triple inference speed. That could raise throughput to 30–45 FPS or more, effectively maxing the camera. - If the network became a bottleneck at higher FPS or resolution, we could move to H.264 video streaming. However, that introduces decoding latency on server and complicates pairing of frames to results (since it becomes a continuous stream). Still, if needing HD resolution, a video stream might be better than many JPEGs. - On the Pi, using the newer camera (HQ camera or Pi Camera Module 3) could allow higher resolution or better image quality, though that might mostly impact accuracy rather than speed. - If multiple cameras needed to be offloaded, we'd need to benchmark how the server scales with parallel streams. We might run separate processes for each stream or a combined scheduler.

In conclusion, the performance results validate that our offloading approach meets the design goal: enabling near-real-time semantic segmentation on a low-power client by leveraging an edge server. The system achieves an order-of-magnitude better frame rate than local processing on the Pi, with latencies in the range suitable for interactive or real-time robotics tasks. The asynchronous pipeline effectively utilizes resources on both ends without undue delay. The Raspberry Pi and laptop each handle the aspects they are best at, and the communication overhead is kept minimal. These findings reinforce the viability of edge offloading for heavy AI tasks in robotics, as has been suggested by other researchers[16][3]. In the next section, we discuss the limitations of our current system and how we plan to address them in future iterations.

Limitations and Future Directions

While the developed system demonstrates substantial benefits, it also has certain limitations that must be acknowledged. In this section, we discuss these limitations and outline potential future directions to enhance the system's robustness, scalability, and capability.

1. Network Dependency and Reliability: The most apparent limitation is the reliance on a stable network connection between the client and server. If the connection is lost or experiences high latency, the system's performance degrades or even halts. In safety-critical or autonomous scenarios, losing the segmentation feed could be dangerous. Our system does not currently have a fail-safe for network outages; the Raspberry Pi would simply stop receiving updates and, in worst case, block on sending new frames. For future improvement, we consider implementing a dual-mode operation:

local fallback and **network monitoring**. As suggested in literature[8][17], one approach is to have a lightweight model on the Raspberry Pi that can run at low frame rate as a backup. The Pi could continuously monitor the round-trip time (or use a heartbeat message) and if the server doesn't respond for, say, a certain duration, the client would switch to this local model or a safe state (e.g., stop the robot or reduce speed) until connectivity is restored. Another approach is to use an alternate communication path (if available) as backup – for example, if Wi-Fi fails, maybe a secondary radio or even 4G connection could be used at reduced quality. Additionally, introducing some basic **fault tolerance** on the server side (like the server saving the last known good result or partial processing if connection cuts mid-frame) could be beneficial.

2. Latency for Highly Dynamic Scenarios: The ~80–100 ms latency, while small, might be problematic for extremely dynamic tasks (e.g., a drone dodging fast objects, or controlling fast actuators). If the environment or camera viewpoint changes drastically within the latency window, the segmentation result might be outdated by the time it's used. Future work could reduce latency further by: - Using even more optimized communication (e.g., UDP with a custom protocol to shave off a few milliseconds of TCP overhead, at the expense of handling packet loss manually). - Reducing processing time via quantized models or simpler models (OpenVINO supports INT8 quantization which could cut inference time by ~30%). - Pipelining the processing of parts of the model: for instance, a concept of **split inference** where some initial layers run on the Pi (to reduce data size by extracting features) and the rest run on the server. This is an area of active research (sending deep feature maps instead of raw images to reduce bandwidth, albeit requiring more compute on client). - Prediction or extrapolation: the client could predict the next segmentation or state based on previous frames (some Kalman filter or optical flow to move segmentation regions slightly) to account for motion during the latency. This is complex and application-specific, but possible.

3. Scalability to Multiple Clients or Cameras: Currently, one server laptop is devoted to one stream from one Pi. In a more complex system, one might have multiple cameras on the same robot (front, rear, etc.), or multiple robots each streaming to a central server. Scaling in either dimension poses challenges. The server's processing power would need to increase proportionally (or the frame rate per stream would drop). The network could also become a bottleneck if many streams share the same bandwidth. A future direction is to incorporate a **multi-client architecture**, possibly leveraging a publish/subscribe model or an edge computing cluster. For instance, if multiple Pis send to one server, a dispatcher could allocate tasks to different cores or even different machines in a cluster. Another concept is **edge cloud** – using a local cloudlet to distribute workload. Software-wise, using something like ROS2 with DDS or an IoT message broker could manage multiple data sources better than custom point-to-point sockets. However, maintaining the low latency would be key; we might explore zero-copy transports for multi-stream. If multiple cameras are on one robot, the system could compress or synchronize their frames together (like sending a stereo pair in one message) to reduce overhead.

4. Model Generality and Upgrades: The segmentation model we used was pre-trained on a specific set of classes. If the task changes (e.g., different environment or objects of interest), one would need to retrain or swap the model. OpenVINO makes it easy to load a new model, but our current code doesn't have a hot-swap mechanism. A future improvement could be to allow the server to host multiple models and let the client specify which one to use for a frame or sequence. For example, a robot could choose a "indoor segmentation" vs "outdoor segmentation" model depending on context. This would also entail a **handshake protocol** where the server informs the client what models or networks it can run (maybe via a model ID), and the client can request accordingly. We could also incorporate model update capabilities: if a better model is available, the server could seamlessly switch after finishing current frames.

5. Security Concerns: Our communication is unencrypted and unauthenticated. In a closed lab network, this was fine, but in real deployments it presents security risks. An attacker could potentially sniff the image data or even inject false results if they hijack the connection. Future iterations should consider using encryption (TLS over TCP or a VPN). However, encryption can add CPU overhead and latency, so it's a trade-off. Modern CPUs (even Pi 4) have hardware acceleration for AES, so TLS might be feasible at our data rates without much impact. Authentication (ensuring only authorized clients/servers connect) is also important for safety. This could be done with pre-shared keys or certificates. We could integrate our protocol into an existing secure framework (like running the whole thing through an SSH tunnel or using a library like OpenSSL to wrap the socket). These additions would harden the system for real-world usage.

6. Energy Consumption: Offloading shifts computational energy usage from the robot (Pi) to the server. The Pi's CPU usage dropped significantly, which means the Pi runs cooler and on lower power (important if on battery). The laptop obviously uses more energy for the heavy compute. In a scenario where the edge server is also battery-powered (e.g., an AGV carrying a compute module), energy might be a concern. Future directions could investigate **energy-aware offloading** – e.g., only offload when necessary. If the segmentation isn't needed continuously, the client could send frames at a lower rate or even request the server to pause to save energy. Also, dynamic adjustment of frame resolution or model complexity could save energy when full detail isn't needed. The Pi 4 has a PoE HAT in our setup, so it was line-powered; if it were on a mobile robot with battery, prolonging battery by offloading (and maybe using a more efficient server or one with a large battery) would likely still be beneficial as many studies suggest overall energy can be reduced by edge offloading[18] (since the server might be more energy-efficient per inference than the Pi struggling at 100% CPU).

7. Timing and Synchronization: In some robotics systems, sensor data from multiple sources must be fused under a common timestamp. Offloading can introduce slight timing unpredictability (network jitter means frame results don't come at perfectly regular intervals). Our system didn't integrate with other sensors like IMU or LiDAR, but if it did, we'd need to synchronize. A future improvement is to timestamp each frame at capture and carry that timestamp through to the result. That way, if the robot is fusing, say, odometry data at certain time points with the segmentation, it knows the exact capture

time of each segmentation result. In our current output, we are effectively assuming the segmentation corresponds to “now” minus latency – which is fine for moderate speeds, but not precise. Integrating with frameworks like ROS would inherently solve some of this (as it has time stamps in messages).

8. Extend to Other Vision Tasks: We focused on semantic segmentation, but the architecture could apply to other heavy vision tasks: object detection, depth estimation, SLAM mapping, etc. Each might have its own data format. For example, an object detector could send back bounding box coordinates instead of a full image mask. A future extension is to support multiple vision services on the same pipeline. We could run an object detection model on the server in parallel and send both segmentation and detection results. This would be useful if the robot needs both (say, segmentation for ground and obstacles, plus detection for specific objects like humans or markers). The protocol can be extended with new message types or combined outputs (e.g., sending a JSON or binary blob of detections along with segmentation). We deliberately kept the design modular so that additional algorithms could plug in – e.g., add another processing stage in the server for another model, or alternate tasks per frame (one frame do segmentation, next frame do detection, etc., interleaving to save compute if needed).

9. Improving BEV and Fusion: Our BEV projection is rudimentary and relies on a flat-ground assumption. In complex 3D environments, this breaks down (e.g., going up ramps, or multi-level structures). Future work could integrate depth sensors or stereo cameras so that a real 3D point cloud can be used to project segmentation (like in AR, coloring a point cloud by segmentation). That would yield a more accurate BEV representation of uneven terrain. Also, to handle vertical information (like overhangs or tall obstacles), a multi-layer map might be needed rather than a single ground plane map. For fusion logic, one could incorporate filtering techniques – e.g., a Bayesian filter that updates the occupancy probabilities with each new segmentation, taking into account possible false positives/negatives from the network to be more robust.

10. User Interface and Feedback: In an operational setting, one might want real-time feedback of system status (e.g., how many frames are in flight, current latency). We envision adding a small telemetry channel either within our protocol or separate, where the server periodically sends status info to the client (or operator’s console). This can help in tuning and trust. For example, if latency spikes, the system could alert or automatically adjust frame rate.

11. Experimenting with Cloud Offloading: While our focus is edge (local server), a curious future experiment is to offload to a remote cloud service (like an AWS EC2 running OpenVINO, or a cloud GPU with a larger model) to see how much the increased network latency (likely ~50ms extra if nearby cloud, more if far) affects things. This would test the feasibility of cloud robotics for segmentation if edge is not available. We suspect for our real-time goal, remote cloud is not ideal unless perhaps 5G ultra-reliable low-latency networks are used. Still, it may be interesting to quantify the trade-offs.

12.Academic Contribution and Generalization: On the research side, one future direction is to formally analyze the system's performance model. We can derive formulas for throughput and latency as functions of processing time, network bandwidth, number of workers, etc., to guide designs in other settings. This could be presented as a generalizable solution for edge offloading in robotics, contributing to the framework of edge robotics design patterns[7]. Additionally, evaluating the system in various scenarios (outdoor, multiple object densities, etc.) would help generalize the findings.

In conclusion, while our system meets its objectives within a controlled environment, real-world deployment will require addressing network robustness, security, and scalability. The promising results encourage further development to make the system production-ready. With planned enhancements in connectivity handling, multi-client support, model flexibility, and integration of additional sensors and tasks, we aim to evolve this work into a comprehensive edge computing solution for robotics vision. By tackling these future directions, we hope to contribute to making advanced perception accessible on even the smallest of robots through intelligent offloading and coordination with edge servers.

Conclusion

We have presented a comprehensive system for offloading semantic segmentation from a low-power embedded client to a more powerful OpenVINO-based server, enabling near real-time performance that would be unattainable on the embedded device alone. The system architecture combines an efficient asynchronous pipeline, a custom binary communication protocol, and optimized inference on the server to achieve a substantial improvement in frame rate and responsiveness for pixel-wise scene understanding on a Raspberry Pi class device. Key features of the system include the parallel operation of camera capture and neural network processing, the use of unique frame IDs to pair out-of-order responses with requests, and multiple result formats (raw labels vs visual overlays) to cater to different application needs.

In our evaluations, the offloaded approach delivered an order-of-magnitude speed-up in throughput (15 FPS vs ~1–2 FPS) and kept end-to-end latency around 0.1 seconds, which is acceptable for many robotics applications. This was achieved while significantly offloading the computational burden from the Raspberry Pi, allowing it to perform other tasks or operate within a lower power envelope. The use of the OpenVINO toolkit on the server proved effective in maximizing the use of available CPU hardware, confirming that specialized optimization libraries can play a crucial role in edge computing setups[5].

The development of the communication protocol was integral to the system's success. By designing a lightweight, length-prefixed message format with minimal overhead, we ensured that streaming high-frequency image data and results did not become a bottleneck. The protocol's incorporation of frame IDs and message types allowed our system to function asynchronously and robustly, a design that could be extended to other remote perception tasks. Our approach demonstrates that a well-designed custom

protocol can outperform more generic solutions in specialized cases, providing reliability and efficiency needed for real-time robotics.

We also introduced postprocessing enhancements such as BEV projection of segmentation and “flower probes”, which bridge the gap between raw pixel predictions and actionable spatial information. These techniques, though not the core focus of offloading, illustrate the additional value that an edge server can provide: not just computing results faster, but computing *richer* results by integrating data and performing additional processing that would be too expensive on the client. By converting the segmentation into a bird’s-eye representation and fusing results over time, the system moves towards a higher-level understanding useful for navigation and planning[14]. This underscores an important point: offloading is not only about doing the same thing remotely, but often enables more sophisticated processing pipelines that enhance overall capabilities.

The work reported in this paper fits into the broader context of edge robotics and distributed intelligence. It provides a case study exemplifying how the synergy of 5G/fast networks and advanced AI toolkits can elevate the abilities of small robots[3]. The trend in robotics and IoT is clearly moving towards collaborative computing, where resource-intensive tasks are dynamically assigned to edge nodes or cloud resources to meet real-time constraints[16]. Our system is a tangible realization of these concepts, applied to the domain of semantic segmentation. We anticipate that similar patterns will emerge for other tasks, and lessons learned here (for instance, about asynchronous design and protocol efficiency) will inform those developments.

There are, of course, areas to refine and build upon. We discussed limitations in the previous section, including network resilience and multi-client scaling. Addressing these will be important for deployment in diverse environments. Nonetheless, the positive results obtained give confidence that with further engineering, the system could be deployed on actual robots (e.g., a Pi-based rover with a companion computing unit) to enable autonomous navigation with rich visual perception. The modular design of our system means pieces of it can be improved independently: for instance, swapping in a more powerful server or a better model would directly translate to better performance without altering the overall framework.

In closing, this work demonstrates that **offloaded asynchronous semantic segmentation** is a viable and effective strategy for embedded robotics. By smartly partitioning the workload between a client and server, we leverage the strengths of each: the client’s proximity to sensors and low-level control, and the server’s computational horsepower and advanced algorithms. The outcome is a system that achieves what neither could alone – high-speed, high-quality scene understanding on a resource-constrained platform. We believe this approach will become increasingly relevant as networked robots and IoT devices proliferate, and we hope that the insights and results we have shared will contribute to the adoption and further innovation of edge-assisted perception in the field.

[1] [5] [6] [12] [13] OpenVINO, OpenCV, and Movidius NCS on the Raspberry Pi - PyImageSearch

<https://pyimagesearch.com/2019/04/08/openvino-opencv-and-movidius-ncs-on-the-raspberry-pi/>

[2] [3] [10] [16] [18] An Improved Soft Actor–Critic Task Offloading and Edge Computing Resource Allocation Algorithm for Image Segmentation Tasks in the Internet of Vehicles

<https://www.mdpi.com/2032-6653/16/7/353>

[4] Asynchronous Inference with OpenVINO™ — OpenVINO™ documentation

<https://docs.openvino.ai/2023.3/notebooks/115-async-api-with-output.html>

[7] [8] [9] [14] [17] Edge Computing and its Application in Robotics: A Survey

<https://arxiv.org/html/2507.00523v1>

[11] Building a frontend for a Raspberry Pi Machine Learning Server with Intel Movidius NCS | by Alexey Stern | TDS Archive | Medium

<https://medium.com/data-science/building-a-frontend-for-a-raspberry-pi-machine-learning-server-with-intel-movidius-ncs-cb940d75103>

[15] BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye ...

<https://research.nvidia.com/labs/eai/publication/bevfusion/>